Autoresearch findings

Kodakodapa

Abstract

This report collects the findings: A census of modal agents in the prisoner's dilemma; When graded PrudentBots cooperate with graded FairBots; An unexploitable modal agent that defects against itself; Families of unexploitable agents that defect against themselves; A certified map of variants of Gödel's ontological argument; Settling informal claims about modal agents in Chicken; Modal agents in the game of Chicken; Three-player modal agents in a public-goods game; Blending self-defecting modal agents with FairBot: exact laws; How large must a program be to tell graded FairBots apart?; Unexploitable modal agents that cooperate with themselves: a census; Round robins of modal agents in the Stag Hunt; Safe modal agents that coordinate in the Stag Hunt: size 13 is the minimum; Nominal-free proofs of the theorems Benzmüller's detector flags.

1 A census of modal agents in the prisoner's dilemma

Setting

Two programs play a one-shot prisoner's dilemma, and each can read the other's source code. Following Barasz et al. [1], a *modal agent* decides by asking what can be proved about the opponent, written with the provability box $\Box$ of the logic GL: p means "the opponent cooperates with me", q "the opponent cooperates with DefectBot". We take the language L of all formulas built from p, q, $\bot$, $\neg$, $\wedge$, $\vee$ and $\Box$ with every atom under a box, measure size as the number of nodes, and identify agents that behave identically against every opponent. Payoffs are the standard 5 (temptation), 3 (mutual cooperation), 1 (mutual defection), 0 (sucker). An earlier finding on this site describes the self-defecting families found in this census; this one reports the census itself.

Result

- **Counts.** Up to size 9 there are 2815 behaviourally distinct agents (new per size 1 to 9: 1, 4, 8, 12, 31, 98, 255, 646, 1760), and every pairwise outcome is solved exactly.

- **No evolutionarily stable agent.** Up to size 9, 180 of the 1335 observational classes are symmetric Nash equilibria, but none is an ESS. Every self-defecting equilibrium is invaded by FairBot, an argument that holds for every modal agent, not only those in the census. Under replicator dynamics from the uniform start, every surviving agent cooperates with itself and the mean payoff tends to 3.

- **The round-robin winner never cooperates.** Up to size 9 the top score belongs to agents that act exactly like DefectBot. Up to size 11 the top 16 agents defect against everyone yet beat DefectBot, because more opponents cooperate with them (9863 against 9688); for example $\Box\Box p \wedge \neg\Box(p \vee \Box\bot)$.

- **Self-defecting cooperators are almost always exploitable.** Among agents that defect against themselves but cooperate with FairBot and CooperateBot (2, 2, 4, ..., 2468, 7032 per size 4 to 14), only 3, 19 and 75 at sizes 12, 13 and 14 are unexploitable by every modal opponent. Each of the others is exploited by one of four simple agents: $\Box\neg p$, $\Box p$, $\Box q$ or $\Box\bot$.

- **A threshold law, and where it breaks.** Up to size 14, every self-defecting agent that cooperates with FairBot cooperates with graded FairBots $F_k = \Box^k p$ exactly for $k \leq a$ and with graded PrudentBots P_i exactly for $i \geq b$, for some $a < b$. The law fails at size 17: the agent $Y = \Box(\Box\Box\bot \vee (p \wedge (q \vee \neg\Box(q \wedge \Box\Box\Box\bot))))$ cooperates with F_1 and F_k for $k \geq 4$, but not with F_2 or F_3.

Why it matters

The literature on modal agents studies a handful of named agents (FairBot, PrudentBot, CliqueBot and a few more) [1, 20]. A census makes the space itself visible: it shows that program equilibrium among modal agents has no evolutionarily stable point, that a round robin rewards agents that never cooperate, and that self-defecting cooperators are rarely safe, with four tiny agents enough to expose all the others.

Proved vs checked

Every outcome is computed exactly in the height semantics of closed GL fixed points (see [11]; the link to provability in arithmetic is Solovay's theorem [24]) and re-checked on large samples by an independent solver; the datasets are pinned by hash. The counts, equilibria and exploitability results hold for all agents up to the stated sizes, not in general. The FairBot invasion argument and the counterexample Y hold in general.

We believe this result is new. Our prior-art searches (2026-10-17 to 2026-10-19) covered:

- OEIS (oeis.org), searched for the census counts 1, 4, 8, 12, 31, 98, 255, 646, 1760 (per size) and 1, 5, 13, 25, 56, 154, 409, 1055, 2815 (cumulative): no match.
- Barasz et al. 2014 (arXiv 1401.5577); LaVictoire et al. 2014; LessWrong / Arbital "Modal combat"; Mennen 2017; Critch, Dennis & Russell 2022 (arXiv 2208.07006); github.com/machine-intelligence/provability.
- Barasz et al. 2014, Robust Cooperation in the Prisoner's Dilemma (arXiv 1401.5577), full text: named agents CB, DB, CliqueBot, FairBot, PrudentBot, TrollBot, JustBot, WaitFairBot_K; Theorems 3.2, 4.8, 4.10; obstacles section.
- LaVictoire et al. 2014, Program Equilibrium in the Prisoner's Dilemma via Löb's Theorem.
- LessWrong post on Barasz et al. and its comments (orthonormal, cousin_it, Will_Sawin, Karl, dankane).
- Arbital / LessWrong wiki "Modal combat".
- Garrabrant, "Prisoners' Dilemma with Costs to Modeling"; Garrabrant, "Modal SAT: Introduction" (2015).
- Mennen, "Modal Combat for games other than the prisoner's dilemma".
- Critch, Parametric Bounded Löb's Theorem and Robust Cooperation of Bounded Agents (arXiv 1602.04184), first sections.
- Searches for round-robin, tournament, replicator or ESS analyses of modal agents: only iterated-PD and FDT evolution papers (e.g. arXiv 2005.05154, no modal agents).
- github.com/machine-intelligence/provability (agents folder contents not visible).
- Barasz et al. 2014 (arXiv 1401.5577), full text; LaVictoire et al. 2014 (Program Equilibrium via Löb's Theorem).
- LessWrong post on Barasz et al. and comments; Arbital "Modal combat".
- Garrabrant, "Prisoners' Dilemma with Costs to Modeling" and "Modal SAT: Introduction"; Mennen, "Modal Combat for games other than the prisoner's dilemma".
- Critch, Parametric Bounded Löb (arXiv 1602.04184), first sections.
- Oesterheld 2022, A Note on the Compatibility of Different Robust Program Equilibria of the Prisoner's Dilemma (arXiv 2211.05057).

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was checked by script; the proof and check files are included so anyone can re-run them.

Claim K1

`In L (atoms p,q each under a box; F,~,&,|,[]; size=node count) agents up to size 9 form 2815 classes of equivalence on finite linear GL models (new per size:  1,4,8,12,31,98,255,646,1760); every pairwise outcome is solved exactly.`

Status: derived.

Verified.

- `checks/K1-c046_census9.py`: class counts per size 1..9; pinned dataset equals rebuilt class list; 18890 games (6 named agents' full rows/columns + 2000 random) agree with an independent height-table solver — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K12

`In L up to size 9 (2815 classes, payoffs 5/3/1/0):  the round-robin maximum 8083 is reached only by 63 agents acting exactly as DefectBot; of 1335 behaviour classes 180 are symmetric Nash (78 self-defecting, all invaded by FairBot) and none is an ESS.`

Status: derived.

Verified.

- `checks/K12-c046_structure9.py`: sha-pinned census9 matrix: round-robin max 8083 by exactly 63 DB-identical agents, second 7979; 1335 observational classes; 180 Nash (78 defect, 102 coop); FB invades all 78; each of 102 has a neutral mutant (CB for 95); no ESS; 183965 used/random games re-solved by independent height-table solver — pass
- `checks/K12-c046_replicator9.py`: pinned census9; discrete replicator, uniform over 2815 chain classes, 20000 steps (exact 1335-class reduction verified 300 steps <1e-12): mean payoff 3 within 1e-6, 788 survivors >1e-6, all self-cooperating, 0.38+-0.005 weight on exploitable classes, final vector equals embedded pin within 2e-6 — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K15

`In L up to size 11 (20441 classes, payoffs 5/3/1/0): round-robin max 59893 is reached only by 16 agents of size 10-11 that defect against all, e.g. []p&~[](p|[]F); DefectBot scores 59193 (rank 21). 1120 symmetric Nash classes, no ESS.`

Status: derived.

Verified.

- `checks/K15-c046_structure11.py`: sha-pinned census11 agents+matrix: rr max 59893 by exactly 16 total defectors of sizes 10,11, each cooperated with by 9863; DB 59193, 9688, 20 above; T vs []F|~[]~p = D/C; 9095 obs classes; Nash 1120 (401 FB-invaded, 719 with neutral mutant); ESS none; 53938 cells re-solved independently — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K17

`In L up to size 14, of the agents that defect vs themselves but cooperate with FairBot and CooperateBot (2,2,4,...,2468,7032 per size 4..14), 3/19/75 at sizes 12/13/14 are unexploitable by every modal opponent; each other is exploited by []~p, []p, []q or []F.`

Status: derived.

Verified.

- `checks/K17-c046_dichotomy14.py`: census counts to size 14 (..,36062,100823,279115); combination counts per size 4..14; machine criterion U holds for exactly 3,19,75 members at sizes 12,13,14 (sha256-pinned list incl. W) and none below; every non-U member exploited by one of []~p,[]p,[]q,[]F; independent height-table solver re-checks 400 witnesses and all 97 U agents' properties; random-trace test finds no exploit for U agents and finds one for 40/40 non-U controls — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K19

`In L up to size 14 (and 15, unchecked run), every self-defecting agent cooperating with FairBot cooperates with F_k (k<=8) iff k<=a and with P_i iff i>=b, a<b. Fails at size 17: Y=[]([][]F|(p&(q|~[](q&[][][]F)))) coops F_1, F_k k>=4, P_i i>=3, not F_2,F_3.`

Status: derived.

Verified.

- `checks/K19-c046_segment.py`: census counts to 14; law (initial F-segment, final P-segment, a<b) on all 9979 S=0 FB-cooperating classes; Y size 17, S=0, CC FB/CB/F_1/F_4..F_12/P_3..P_12, DD with F_2,F_3,P_0..P_2, D vs DB, criterion U; independent string solver for all Y rows; 3000 random traces; GLS certificates with flipped controls for Y vs Y,F_1..F_4,P_2,P_3,CB — pass

Prior art. Possibly new: no prior art found in the owner's search.

Discussion

The census is exhaustive only up to the stated sizes (9 for the counts and equilibria, 11 for the round robin, 14 for exploitability and the threshold law), and only for the language L with the single probe "the opponent cooperates with DefectBot". Richer probes (how the opponent treats FairBot or CooperateBot) would give a different, larger space, and the counts depend on the exact choice of connectives and size measure. Results about tournaments and replicator dynamics depend on the payoffs and on the uniform starting population; we report them as properties of this population, not of modal agents in general. The FairBot invasion argument and the counterexample Y are the exceptions: they hold for all modal agents.

Two questions stay open. First, whether the unexploitable self-defecting agents found up to size 14 all belong to known families: some of them do, and the rest were shown different from family members of small index but not classified. Second, whether the threshold law, which fails at size 17, holds in a weaker form, for example for all agents of a given family. The natural next searches are the same census for other two-action games (Chicken, Stag Hunt), where outcomes are unchanged and only the scoring differs, and a census with more probes.

Provenance. Run `20261017-1126`, claims K1, K12, K15, K17, K19, run-folder commit `662166ecff0651902cc5fcb04bf747c62d795af2`, published 2026-10-19.

2 When graded PrudentBots cooperate with graded FairBots

Setting

Two programs play a one-shot prisoner's dilemma, and each can read the other's source code. A *modal agent* [1] decides by asking what Peano arithmetic (PA) can prove about the opponent, written with the provability box $\Box$ of the logic

GL. Here p means "the opponent cooperates with me" and q means "the opponent cooperates with DefectBot", the agent that always defects. FairBot, $\Box p$, cooperates exactly when it can prove that the opponent cooperates with it. PrudentBot adds a safety test: it also requires a proof, in the stronger system PA+1, that the opponent would not cooperate with DefectBot, so it does not reward an opponent that cooperates with everyone.

We study two graded families. The graded FairBot $F_k = \Box^k p$ $(k \geq 1)$ asks for k nested layers of provability; F_1 is FairBot. The graded PrudentBot $P_j = \Box p \wedge \Box(\neg\Box^j \bot \to \neg q)$ $(j \geq 0)$ runs its safety test in PA+j, since $\neg\Box^j\bot$ expresses the consistency of PA+$(j-1)$; P_1 is PrudentBot.

Result

P_j and F_k cooperate with each other exactly when $k \leq j$; when $k > j$, both defect. In words: a PrudentBot that runs its safety test in a stronger system can afford to cooperate with FairBots that demand more nested proof, and the exchange rate is one for one. Every P_j still defects against CooperateBot, so the extra cooperation costs nothing in safety.

Why it matters

Barasz et al. remark that PrudentBot's safety test may use a stronger system than PA+1 and that this widens the set of agents it cooperates with, and online discussion of the paper looked at specific pairs of such variants. Here the relation is pinned down exactly for the two graded families, for all indices: the boundary of cooperation is the single inequality $k \leq j$.

Proved vs checked

Proved for all $j \geq 0$ and $k \geq 1$ in Lean 4, in the height semantics of closed GL fixed points (on the chain of heights, both agents cooperate at every height when $k \leq j$; when $k > j$ both defect above height $j+k$). The link to provability in PA is Solovay's theorem [24] (see also [11]), cited, not formalised, and the reading in the standard model assumes PA is sound.

This result extends Barasz et al. (arXiv 1401.5577), remark after Theorem 3.2: PrudentBot's DefectBot check may use PA+n for larger n; LessWrong comments on "Robust Cooperation in the Prisoner's Dilemma" (orthonormal, cousin_it): indexed Prudent(m,n)Bot variants with specific pairs. Our prior-art search (2026-10-21) covered:

- Barasz et al. 2014, Robust Cooperation in the Prisoner's Dilemma (arXiv 1401.5577), full text: agents CB, DB, CliqueBot, FairBot, PrudentBot, TrollBot, JustBot, WaitFairBot_K; Theorems 3.2, 4.8, 4.10 and the obstacles section.
- LaVictoire et al. 2014, Program Equilibrium in the Prisoner's Dilemma via Löb's Theorem (intelligence.org/files/ProgramEquilibrium.pdf).

- LessWrong post “Robust Cooperation in the Prisoner’s Dilemma” and its comments (orthonormal, cousin_it, Will_Sawin, Karl, dankane).
- Arbital / LessWrong wiki “Modal combat”.
- Garrabrant, “Prisoners’ Dilemma with Costs to Modeling” (Alignment Forum).
- Mennen, “Modal Combat for games other than the prisoner’s dilemma” (agentfoundations.org item 1304).
- github.com/machine-intelligence/provability (Haskell solver and combat code; the agents folder contents were not visible).

If you know of related work we missed, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was checked by scripts that run Lean proofs; the proof and check files are included so anyone can re-run them.

Claim K8

`For all j>=0, k>=1 (height-chain semantics of closed GL fixed points): P_j=[]p&[](~[]^jF->~q) and F_k=[]^k p cooperate at every height iff k<=j; if j<k both defect above height j+k. Lean-proved; P_j defects vs CB.`

Status: derived ů author check only.

Verified.

- `checks/K8-c046_pjfk_lean.py`: embedded Lean 4 proof of pj_fk and pj_cb compiles with no sorryAx; flipped-conclusion variant is rejected by Lean; explicit height table for j<=8,k<=8 matches (coop everywhere iff k<=j, else both defect above j+k with exact threshold) — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Provenance. Run `20261017-1126`, claims K8, run-folder commit `662166ecff0651902cc5fcb04bf747c62d79` published 2026-10-18.

3 An unexploitable modal agent that defects against itself

Setting

Two programs play a one-shot prisoner’s dilemma, and each can read the other’s source code. Following Barasz et al. [1], a *modal agent* decides by asking what

can be proved about the opponent in Peano arithmetic, written with the provability box $\Box$ of the logic GL. FairBot, $\Box p$, cooperates exactly when it can prove that the opponent cooperates with it (p); CooperateBot always cooperates; DefectBot always defects. An agent is *unexploitable* if no modal opponent ever gets it to cooperate while the opponent defects. Graded FairBots are $F_k = \Box^k p$ and graded PrudentBots are $P_j = \Box p \wedge \Box(\neg\Box^j \bot \to \neg q)$. We consider agents built from p, from q ("the opponent cooperates with DefectBot"), $\bot$, $\neg$, $\wedge$, $\vee$ and $\Box$, with every atom under a box, and measure size as the number of nodes in the formula.

Result

The agent $W = \Box(\Box\Box\bot \vee (p \wedge (q \vee \neg\Box q)))$ is unexploitable, cooperates with FairBot and with CooperateBot, yet defects against an exact copy of itself. An exhaustive search shows that no agent of size 11 or less has this combination of properties, and W (size 12) is one of the three smallest. It is the first member of an infinite family $W_n = \Box(\Box^n\bot \vee (p \wedge (q \vee \neg\Box q)))$, $n \geq 2$, all with the same four properties, with exact laws for how W_n plays against graded FairBots $\Box^k p$, graded PrudentBots and other members of the family.

Why it matters

Every unexploitable agent in the literature we checked (FairBot, PrudentBot, CliqueBot) cooperates with its own copy. W_n shows that unexploitability, cooperation with FairBot and generosity towards CooperateBot do not force self-cooperation: the delayed "gate" $\Box^n\bot$ lets the agent tolerate FairBot's failure to prove anything about DefectBot, while two copies of W_n end in mutual defection (both defect above height $n + 1$).

Proved vs checked

The properties of W_n and its laws against graded FairBots, graded PrudentBots and other W_m are proved for all indices in Lean 4 (height semantics of closed GL fixed points; the link to provability in arithmetic is Solovay's theorem [24], cited, not formalised; see also [11]). The minimality of size 12 is an exhaustive computation over all agents up to size 12 in the stated language.

We believe this result is new. Our prior-art search (2026-10-17) covered:

- Barasz et al. 2014, Robust Cooperation in the Prisoner's Dilemma (arXiv 1401.5577), full text: agents CB, DB, CliqueBot, FairBot, PrudentBot, TrollBot, JustBot, WaitFairBot_K; Theorems 3.2, 4.8, 4.10 and the obstacles section.
- LaVictoire et al. 2014, Program Equilibrium in the Prisoner's Dilemma via Löb's Theorem (intelligence.org/files/ProgramEquilibrium.pdf).

- LessWrong post "Robust Cooperation in the Prisoner's Dilemma" and its comments (orthonormal, cousin_it, Will_Sawin, Karl, dankane).
- Arbital / LessWrong wiki "Modal combat".
- Garrabrant, "Prisoners' Dilemma with Costs to Modeling" (Alignment Forum).
- Mennen, "Modal Combat for games other than the prisoner's dilemma" (agentfoundations.org item 1304).
- github.com/machine-intelligence/provability (Haskell solver and combat code; the agents folder contents were not visible).

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was partly checked by scripts that run Lean proofs (claims K9, K11) and partly checked by script (claims K6); the proof and check files are included so anyone can re-run them.

Claim K6

`In L (atoms p,q boxed; F,~,&,|,[]; size=nodes), the smallest agent that cooperates with FairBot and CooperateBot, defects against itself and is unexploitable has size 12; W=[]([][]F|(p&(q|~[]q))) is one, unexploitable by every opponent; none of size <=11.`

Status: derived ů author check only.

Verified.

- `checks/K6-c046_combo12.py`: census counts to size 12; S=0,FBc=1,CBc=1 counts per size 4..12 = 2,2,4,8,19,46,120,357,900; none <=11 and exactly 3 of size 12 unexploited by all 2815 size<=9 classes; W's self/FB/CB/DB outcomes and zero exploiters by an independent height-table solver; GLS certificates for W vs W, FB, CB with flipped-candidate controls — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K9

`For every n>=2, the agent W_n=[]([]^n F|(p&(q|~[]q))) of size n+10 is unexploitable by every modal opponent, mutually cooperates with FairBot and CooperateBot, and defects against itself (both copies defect above height n+1); W_0, W_1 fail with FairBot.  Lean-proved.`

Status: derived ů author check only.

Verified.

- `checks/K9-c046_wn_lean.py`: embedded Lean 4 theorems w_gate, w_unexploitable, w_db, w_cb, w_self, w_self_db, w_fb compile with no sorryAx; three false variants rejected; height tables n<=12 and 3000 random opponent traces agree; size(W_n)=n+10 — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K11

`For all n>=0, k>=1, j,m>=0 (height semantics of closed GL fixed points): W_n and F_k cooperate at every height iff k<n; W_n and P_j iff 2<=n<j; W_n and W_m both defect from height n+m+3. Hence all W_n (n>=2) are distinct. Lean-proved.`

Status: derived ů author check only.

Verified.

- `checks/K11-c046_wpairs_lean.py`: embedded Lean 4 theorems w_fk, pj_db, w_pj, w_wm compile with no sorryAx; four false variants rejected with proof errors; height tables n<=12, k,j,m<=14 (heights 0..79) agree with all theorems and K10's n=2..5 predicates — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Provenance. Run `20261017-1126`, claims K6, K9, K11, run-folder commit `662166ecff0651902cc5fcb04bf747c62d795af2`, published 2026-10-18.

4 Families of unexploitable agents that defect against themselves

Setting

Two programs play a one-shot prisoner's dilemma, and each can read the other's source code. Following Barasz et al. [1], a *modal agent* decides by asking what can be proved about the opponent, written with the provability box $\Box$ of the logic GL. p means "the opponent cooperates with me" and q means "the opponent cooperates with DefectBot". FairBot is $\Box p$, graded FairBots are $F_k = \Box^k p$, graded PrudentBots are $P_j = \Box p \wedge \Box(\neg\Box^j \bot \to \neg q)$, and ConBot is $\neg\Box\bot$. An agent is *unexploitable* if no modal opponent ever gets it to cooperate while the opponent defects.

An earlier finding on this site described $W_n = \Box(\Box^n \bot \vee (p \wedge (q \vee \neg\Box q)))$: unexploitable, cooperative with FairBot and CooperateBot, yet defecting against

its own copy. This finding describes four further families of unexploitable agents that defect against themselves, and how they relate.

Result

- **Gate families** $V_{n,r} = \Box(\Box^n \bot \vee (p \wedge (q \vee \neg\Box^r q)))$, which contain $W_n = V_{n,1}$. $V_{n,r}$ and F_k cooperate exactly when $k + r \leq n$; $V_{n,r}$ and P_j exactly when $r < n < j$; any two members defect against each other. For $r < n$ every member is unexploitable, and these members are pairwise distinct.
- **Prudent families** $G_{m,j} = \Box(\Box^m p \wedge (q \vee \Box(\Box^j \bot \vee \neg q)))$ **and** $H_{m,j} = \Box^{m+1}(p \wedge (q \vee \Box(\Box^j \bot \vee \neg q)))$, $m \geq j \geq 1$. Both are unexploitable, cooperate with FairBot and CooperateBot, defect against themselves, and cooperate with F_k exactly when $k \leq j$ and with P_i exactly when $i > m$. So $G_{m,j}$ and $H_{m,j}$ cannot be told apart by graded FairBots or PrudentBots; the agent $S_j = \Box^{j+1}(p \vee \neg\Box^{j+1} q)$ separates them ($H_{m,j}$ defects against it while it cooperates; against $G_{m,j}$ both defect). ConBot separates the two kinds of family: $G_{m,j}$ defects against it, every $V_{n,r}$ with $n \geq 1$ does not.
- **A common family** $J_{a,b,j} = \Box^{a+1}(\Box^b p \wedge (q \vee \Box(\Box^j \bot \vee \neg q)))$, which contains G ($a = 0$) and H ($b = 0$). Checked for $a, b \leq 5$, $j \leq 6$: $J_{a,b,j}$ cooperates with F_k exactly when $k \leq j$ and with P_i exactly when $i > a+b$, and exact laws hold against a two-parameter family of separating probes; the 70 members with $j \leq a + b \leq 5$ are pairwise distinct. The size-13 member $J_{1,1,1}$ has the same outcomes as $G_{2,1}$ and $H_{2,1}$ against $F_1, \ldots, F_4$ and $P_0, \ldots, P_5$, yet two opponents of size at most 9 distinguish all three.

Why it matters

The unexploitable agents we found in the literature (FairBot, PrudentBot and its graded and probabilistic variants [1], [23]) all cooperate with their own copies. These families show that defecting against oneself is not a single odd example: it comes in several infinite families with different mechanisms (a delayed gate on the DefectBot test in V, a PrudentBot-style check in G, H and J). Their behaviour against graded FairBots and PrudentBots is described by simple thresholds, and these opponents alone do not identify an agent: distinct members can agree on all of them and need other probes to be told apart.

Proved vs checked

The laws for $V_{n,r}$, $G_{m,j}$ and $H_{m,j}$ (including the separations by ConBot and S_j) are proved for all indices in Lean 4, in the height semantics of closed GL fixed points; the link to provability in arithmetic is Solovay's theorem [24], cited, not formalised (see also [11]). The statements about $J_{a,b,j}$ and $J_{1,1,1}$ are checked, not proved: exact tables for the stated parameter ranges, with every member of size at most 15 independently certified by GL fixed-point solving.

The unexploitability of $J_{a,b,j}$ is supported by a short argument (the agent implies $\Box^{a+b+1}p$) and randomised safety tests, not by a proof for all indices.

We believe this result is new. Our prior-art searches (2026-10-17 to 2026-10-18) covered:

- Barasz et al. 2014, Robust Cooperation in the Prisoner's Dilemma (arXiv 1401.5577), full text: named agents CB, DB, CliqueBot, FairBot, PrudentBot, TrollBot, JustBot, WaitFairBot_K; Theorems 3.2, 4.8, 4.10; obstacles section.
- LaVictoire et al. 2014, Program Equilibrium in the Prisoner's Dilemma via Löb's Theorem.
- LessWrong post on Barasz et al. and its comments (orthonormal, cousin_it, Will_Sawin, Karl, dankane).
- Arbital / LessWrong wiki "Modal combat".
- Garrabrant, "Prisoners' Dilemma with Costs to Modeling"; Garrabrant, "Modal SAT: Introduction" (2015).
- Mennen, "Modal Combat for games other than the prisoner's dilemma".
- Critch, Parametric Bounded Löb's Theorem and Robust Cooperation of Bounded Agents (arXiv 1602.04184), first sections.
- Searches for round-robin, tournament, replicator or ESS analyses of modal agents: only iterated-PD and FDT evolution papers (e.g. arXiv 2005.05154, no modal agents).
- github.com/machine-intelligence/provability (owner, 2026-10-18): its example agent file (target.ua) and the Haskell agent catalogue (Agents.hs) (CB, DB, FB, FB_1..5, TbF_1..3, FB_r, MB, WB, WB_2, aMB, sMB, IB, PB, PB_2, PB_3, NB, JB, TB, QB_1..3, modal UDT). Evaluated in height semantics (our reading of its [k] and boxk notation as PA+k provability): none of them defects against itself while cooperating with FairBot and CooperateBot; none has the form of V, G, H or J.
- Barasz et al. 2014 (arXiv 1401.5577), full text; LaVictoire et al. 2014 (Program Equilibrium via Löb's Theorem).
- LessWrong post on Barasz et al. and comments; Arbital "Modal combat".
- Garrabrant, "Prisoners' Dilemma with Costs to Modeling" and "Modal SAT: Introduction"; Mennen, "Modal Combat for games other than the prisoner's dilemma".
- Critch, Parametric Bounded Löb (arXiv 1602.04184), first sections.
- Oesterheld 2022, A Note on the Compatibility of Different Robust Program Equilibria of the Prisoner's Dilemma (arXiv 2211.05057).

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was partly checked by scripts that run Lean proofs (claims K14, K18, K21) and partly checked by script (claims K20, K22, K23); the proof and check files are included so anyone can re-run them.

Claim K14

`For all n>=0, r>=1, k>=1, j>=0 (height semantics of closed GL fixed points), V_{n,r}=[]([]^nF|(p&(q|~[]^r q))): CC with F_k iff k+r<=n; with P_j iff r<n<j; any two V's defect. For r<n each is universally unexploitable, all pairwise distinct. Lean.`

Status: derived ů author check only.

Verified.

- `checks/K14-c046_vfam_lean.py`: embedded Lean 4 theorems qclause_r, v_unexploitable, v_db, v_cb, v_fk, pj_db, v_pj, v_vm, v_params compile with no sorryAx; 5 false variants rejected; literal height tables n<=10, r<=5, k,j<=12 and V-vs-V n,m<=6, r,s<=3 agree; 35 r<n agents have distinct F/P outcome vectors; size(V_{n,r})=n+r+9 — pass (Lean-proved, no sorryAx)
- `checks/K14-c046_vfam_lean.py`: embedded Lean 4 theorems qclause_r, v_unexploitable, v_db, v_cb, v_fk, pj_db, v_pj, v_vm, v_params compile with no sorryAx; 5 false variants rejected; literal height tables agree; 35 r<n agents have distinct F/P outcome vectors — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K18

`For all m>=j>=1 (height semantics of closed GL fixed points), G_{m,j}=[]([]^m p&(q|[]([]^jF|~q))), size m+j+10, is unexploitable, coops with FB and CB, defects vs itself; CC with F_k iff k<=j, P_i iff i>m; defects vs ~[]F unlike every V_{n,r} with n>=1. Lean.`

Status: derived ů author check only.

Verified.

- `checks/K18-c046_gfam_lean.py`: embedded Lean 4 theorems gq, g_unexploitable, g_db, g_cb, g_conbot, g_fk, g_pj, g_gg, v_conbot, g_params compile with no sorryAx; 4 false variants rejected; literal height tables m<=8, j<=9, k,i<=11 and G-vs-G m,m'<=5 agree; independent string solver:

self/FB/CB/DB/ConBot for m,j<=3, W_2 coops with ConBot; size(G_{m,j})=m+j+10 — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K20

`For all H_{m,j}=box^{m+1}(p&(q|box(box^j F|~q))), 1<=j<=m, size m+j+10<=15 (six agents), self-play is DD, FB/CB CC; CC with F_k iff k<=j (k<=4), P_i iff i>m (i<=5). GLS certifies 102 cells including separators from matching G and bounded V.`

Status: derived ů author check only.

Verified.

- `checks/K20-outer_prudent_family.py`: Exhausts six H parameters of size<=15; all 102 pinned fixed-point cells pass GLS with mutation controls; common F/P rows distinguish all six; explicit matching-G and ConBot/V separators. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K21

`For m>=1, j>=1 (height semantics of closed GL fixed points), H_{m,j}=[]^{m+1}(p&(q|[]([]^jF|`
`is unexploitable; DB trace h<=m; CC with F_k iff k<=j, P_i iff i>m; H vs H defect (j<=m); S_j=[]^{j+1}(p|~[]^{j+1}q) gives D/C vs H_{m,j}, D/D vs G_{m,j}. Lean.`

Status: derived ů author check only.

Verified.

- `checks/K21-c046_hfam_lean.py`: embedded Lean 4 theorems h_fail, h_unexploitable, h_db, h_cb, h_conbot, h_fk, h_pj, h_hh, s_db, h_sep, g_sep, h_params compile with no sorryAx; 4 false variants rejected; literal height tables m,j<=8, k,i<=11, H-vs-H m,m'<=5, H/G vs S_j agree; independent string solver for m,j<=3: self, FB, CB, DB, ConBot, F_1..4, P_0..5, S_j; size(H)=m+j+10, size(S_j)=2j+6 — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K22

`In L, the size-13 agent J=[][]([]p&(q|[]([]F|~q))) has self DD and FB/CB CC. Its F1..4/P0..5 outcomes match G21 and H21, but two size<=9 opponents distinguish all three; all 48 stated cells are GLS-certified.`

Status: derived ů author check only.

Verified.

- `checks/K22-mixed_prudent.py`: 48 hash-pinned GLS-certified cells with mutation controls; all three agents size13; named rows agree; two small opponents distinguish G21,J,H21. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K23

For $J_{a,b,j} = \Box^{a+1}(\Box^b p \wedge (q \vee \Box(\Box^j F \vee \neg q)))$, $0 \le a, b \le 5$, $1 \le j \le 6$: CC with F_k ($k \le 9$) iff $k \le j$, P_i ($i \le 9$) iff $i > a + b$; vs $S_{u,t}$ ($u, t \le 8$): CC if $u \le j$, else D/C iff $t + b \le u$, else DD. The 70 with $j \le a + b \le 5$ are distinct.

Status: derived ů author check only.

Verified.

- `checks/K23-c046_jfam.py`: literal tables a,b<=5, j<=6: DB/CB/ConBot, F_k (k<=9), P_i (i<=9), S_{u,t} (u,t<=8) laws incl. defection heights, J vs J defect, random-trace safety; 70 members (j<=a+b<=5) have distinct vectors; independent string solver: 882 cells for all 21 members of size<=15 — pass

- `checks/K23-c046_jfam_gls.py`: all 21 J_{a,b,j} with 1<=j<=a+b, size<=15: 265 cells (self, CB, DB, ConBot, F_j/F_{j+1}, P_{a+b}/P_{a+b+1}, S_{j,j+3}, S_{u,t} boundaries t+b=u vs u+1 for u=j+1,j+2, G21/J111/H21 split on S_{2,1},S_{2,2}) match the K23 law formulas and pass GLS fixed-point certificates with flipped-candidate rejection — pass

Prior art. Possibly new: no prior art found in the owner's search.

Provenance. Run `20261017-1126`, claims K14, K18, K20, K21, K22, K23, run-folder commit `662166ecff0651902cc5fcb04bf747c62d795af2`, published 2026-10-18.

5 A certified map of variants of Gödel's ontological argument

Setting

Gödel's ontological argument, in Scott's version, uses a second-order predicate P ("positive") and five axioms: A1 (exactly one of a property and its negation is positive; split here into A1a, "not both", and A1b, "at least one"), A2 (positivity is closed under necessary entailment), A3 ($P(G)$, where G, "God-like", means having every positive property), A4 (positive properties are necessarily positive)

and A5 (necessary existence is positive). The conclusion is that a God-like being exists, $\exists x\, G(x)$, and necessarily exists, $\Box\exists x\, G(x)$. A well-known side effect is *modal collapse*: everything true is necessarily true.

Many variants and weakenings are studied, mostly one variant at a time with an automated theorem prover [8, 4, 2, 6]. This finding maps the space systematically. We take every subset of each variant's axioms and the nine frame classes K, D, T, B, K4, KB, KD45, S4 and S5. We vary the semantics: all intensional properties, rigid properties or extensional properties (Fitting [15]), and constant or varying (actualist) domains. Cell by cell we decide whether the axioms are consistent, whether they prove the conclusion and whether they force collapse.

Result

- **Scott, all properties** (constant or varying domains): the God-like being exists exactly when the subset contains A1a, A2, A3 and A5 and the frames are B, KB or S5; necessary existence also holds on KD45. Collapse holds exactly when the subset contains A1a, A2 and A5, on the same frames. A1b and A4 are never needed. With A1, A2 and A5 alone, every symmetric frame is forced to consist of isolated worlds.

- **Scott, rigid properties:** with constant domains no consistent cell forces collapse, and the conclusion holds on every frame exactly when the subset contains A1, A2 and A3, or A1, A3 and A5 (with Scott's essence). With varying domains existence needs A1a, A2, A3 and A5 on reflexive frames (T, B, S4, S5), or A4 as well on KB; with A4 added, necessary existence holds on every frame.

- **Anderson's variant** (with $H = P(G_A)$): existence needs A1a, A2 and H on B, KB or S5, and on KD45 only with A4 or A1b, and only with constant domains. A5 follows from A2 and H. Adding A1b, the direction Anderson removed, brings collapse back on B and S5, and on KB only together with A4.

- **Fitting's extensional variant** ($H = P(G)$, $N = P(NE)$): with constant domains no cell forces collapse, and the conclusion holds exactly for supersets of {A1a, A2, H} or {A1a, H, N}. With varying domains existence needs all of {A1a, A2, H, N} and reflexive frames (T, B, S4, S5); necessary existence also holds on KD45.

- **Benzmüller's simplified variant** (no positive empty property, closure under entailment, closure under arbitrary meets): every cell is consistent and none collapses; the conclusion holds exactly for the full axiom set with closure under local entailment, and never with closure under necessary entailment alone.

Why it matters

Each of these variants has been studied before, and many single cells of the map are known: the dependence of collapse on symmetry, the sufficiency of KB, the redundancy of A5 in Anderson's variant, the absence of collapse in Anderson's, Fitting's and the simplified variant [8, 4, 9, 2]. What we did not find is the complete picture: for each variant, exactly which axiom subsets and frame conditions give the conclusion or collapse, and the minimal sets. The map shows, for example, that in Scott's version neither A1b nor A4 ever matters, and that with rigid properties and constant domains collapse disappears altogether.

Proved vs checked

Every positive cell is proved in Lean 4 (core library, standard axioms only) in a shallow embedding of higher-order modal logic on arbitrary Kripke frames of the class. Every negative cell has an explicit finite countermodel, re-evaluated by a script. The tables cover the stated axiom sets, semantic choices and nine frame classes; other variants (other definitions of essence, other conjunction principles) are outside the map. We did not include the variants with Gödel's binary conjunction axiom: that it fails on infinite domains is known [3].

This result extends the single cells and per-proof axiom dependencies of Scott's and Gödel's variants established with other provers (Benzmüller & Scott 2025; Benzmüller 2026); the analysis of Anderson's variant and the redundancy of its axioms (Benzmüller, Weber & Woltzenlogel Paleo 2017); Fitting's and Anderson's collapse-free variants (Fitting 2002; Benzmüller & Fuenmayor 2019); Benzmüller's simplified variant (2020). Our prior-art search (2026-10-19) covered:

- Benzmüller & Scott, "Notes on Gödel's and Scott's variants of the ontological argument", Monatshefte für Mathematik (2025), and its Isabelle/HOL development (AFP entry Notes_On_Goedels_Ontological_Argument, theory outline): per-theorem dependencies; modal collapse for Gödel variants 2 and 3 needs "only symmetry"; Scott's T3 and MC use Rsymm; Scott's variant fails in K only in the last step; UniqueEss lemmas (an essence of x is necessarily had only by x).
- Benzmüller, "Gödel's and Scott's Variants of the Ontological Argument in Lean 4 and TPTP THF" (arXiv 2609.26806, Oct 2026): Lean port with `#print axioms` dependency tables (Table 4) over S5, S4 and K embeddings, actualist/possibilist/mixed quantifiers; Th4 of Gödel variant 2 does not need Ax4; Scott's T3 and MC depend on symmetry alone; Gödel's 1970 axioms inconsistent in K.
- Benzmüller, "Proofs Without Nominals: Gödel's Ontological Argument, its Shallow Embedding, and the Open Questions of the Monatshefte Notes" (arXiv 2609.36279, 2026): the conjunction axiom must be emended (two or more conjuncts); collapse in K under the original axiom; in variant 3 an essence of x can only be "being x".

- Benzmüller & Woltzenlogel Paleo, automating Gödel's proof (2014–2016, incl. arXiv 1308.4526): KB suffices; Scott's axioms imply collapse; Gödel's original inconsistent.
- Benzmüller & Fuenmayor, positive properties, ultrafilters and modal collapse (arXiv 1910.08955): Anderson and Fitting variants avoid collapse.
- Benzmüller, "A (simplified) supreme being necessarily exists" (arXiv 2001.04701; KR 2020): simplified axioms valid in K and T without collapse.
- github.com/drewarrowood/godel-ontological (Lean 4): Scott with frame hypotheses; Symmetric gives local T3, S4 countermodel; rigid (world-independent) positivity makes A4 free but admits a countermodel to global T3; Russell-cut table (dropping A5 or A3).
- Fitting, Types, Tableaus and Gödel's God (2002); Sobel, Logic and Theism (2004); Anderson (1990); Wikipedia "Gödel's ontological proof" and "Modal collapse".
- Benzmüller, "A Comment on Modal Collapse and Ultrafilters in Gödel's Ontological Argument" (arXiv 2608.07578, 2026), replying to Odifreddi & Gomes, Logica Universalis (2026).
- Benzmüller, Weber & Woltzenlogel Paleo, "Computer-Assisted Analysis of the Anderson–Hájek Ontological Controversy" (Logica Universalis 2017; summary version read).
- Anderson & Gettings, "Gödel's Ontological Proof Revisited", in Gödel '96 (abstract only).

If you know of related work we missed, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was checked by scripts that run Lean proofs; the proof and check files are included so anyone can re-run them.

Claim K2

Full-property constant-domain embedding, Scott essence: A1, A2, A5 imply $R\,w\,v \leftrightarrow v = w$ and modal collapse on every symmetric frame; adding A3 gives $\exists x\, G(x)$ and $\Box\exists x\, G(x)$ there, and $\Box\exists x\, G(x)$ on euclidean frames. A4 is not used.

Status: derived ů author check only.

Verified.

- `checks/K2-c009_lean_table.py`: lean –stdin exit 0, no sorry, no axiom decls; haec_ess, NE_isolated, A125_isolated_succ, A125_sym_iso, A125_sym_collapse, A1235_sym_god, A1235_euclid_box, majority_model all within {propext, Classical.choice, Quot.sound} — pass

Prior art. Partial prior art.

Claim K7

Constant domain, rigid properties, Scott/Gödel essence, all 32 subsets × 9 classes certified: no consistent cell forces collapse; $\exists x\, G(x)$ and $\Box\exists x\, G(x)$ hold iff the subset contains A1,A2,A3, or A1,A3,A5 with Scott essence, on every frame.

Status: derived ů author check only.

Verified.

- `checks/K7-c009_rigid_table.py`: lean –stdin exit 0, no sorry, no axiom decls; A123_god/box, A135S_god/box, A12_A5S, A125G_no_world within {propext, Classical.choice, Quot.sound}, scott_rigid_noncollapse axiom-free; all 576 cells × 4 outcomes certified by a brute-force re-evaluated model or a Lean theorem whose hypotheses ⊆ cell axioms; no pool model contradicts a Lean cell; stated pattern asserted for every cell — pass

Prior art. Partial prior art.

Claim K9

Constant-domain full-property Anderson slice: $\exists x\, G_A(x)$ holds iff the subset contains A1a, A2, H and the class is B, KB, S5, or KD45 with A4; $\Box\exists x\, G_A(x)$ iff it contains A1a, A2, H on B, KB, KD45, S5. A5 is derivable from A2 and H.

Status: derived ů author check only.

Verified.

- `checks/K9-c009_anderson_open.py`: lean –stdin exit 0, no sorry, no axiom decls; NE_univ, A2H_A5, poss, sym_god, sym_box, eucl_box, eucl_A4_god, kd45_countermodel within {propext, Classical.choice, Quot.sound}; another researcher's verbatim evaluator gives the 3-world KD45 counter-model axiom mask {A1a,A2,H,A5}, no God-like being at the root, box-exists true; A2&H imply A5 on all 4136 models of K8's bound; resolves all 16 open K8 cells — pass

Prior art. Partial prior art.

Claim K13

Constant-domain extensional Fitting, all subsets of A1a,A1b,A2,H,N × nine frame classes: all 288 cells are consistent and permit noncollapse; both existence conclusions hold exactly for supersets of $\{A1a, A2, H\}$ or $\{A1a, H, N\}$. A1b is unnecessary.

Status: derived ů author check only.

Verified.

- `checks/K13-fitting_split.py`: Lean core proves four existence/box theorems with A1a only and NE universal; standard axioms only, no sorry or axiom declarations. Re-evaluates explicit models covering every negative cell and all consistency/noncollapse cells; regenerates pinned 288-cell table. — pass

Prior art. Partial prior art.

Claim K15

Varying-domain (actualist) full-property Anderson, 32 subsets × 9 classes: as in constant domains except $\exists^E x\, G_A(x)$ on KD45 with A4 now fails; a 3-world KD45 model satisfies all five axioms, but its God-like being does not exist at the root.

Status: derived ů author check only.

Verified.

- `checks/K15-c009_varying_anderson.py`: lean –stdin exit 0, no sorry, no axiom decls; god_exists_succ, poss, sym_god, sym_box, eucl_box, A2H_A5, kd45_varying, entails_top within {propext, Classical.choice, Quot.sound}; guarded evaluator = another researcher's verbatim evaluator at E=all on 4136 models; kd45_varying re-evaluated (mask 31, exists false, box-exists true, noncollapse); varying-E sweep of 16528 pairs consistent with all theorems; 288-cell table asserted (pattern, no OPEN, 15/KD45 and 31/KD45 refuted) and sha256-pinned — pass

Prior art. Partial prior art.

Claim K16

Varying-domain (actualist) extensional Fitting, 32 subsets of A1a,A1b,A2,H,N × 9 classes: $\exists^E x\, G(x)$ holds iff the subset contains $\{A1a, A2, H, N\}$ on T, B, S4, S5; $\Box\exists^E x\, G(x)$ also on KD45. K13's sets $\{A1a, A2, H\}$ and $\{A1a, H, N\}$ fail in S5.

Status: derived ů author check only.

Verified.

- `checks/K16-c009_varying_fitting.py`: lean –stdin exit 0, no sorry, no axiom decls; god_nonempty, refl_god, T_god, T_box, eucl_box within {propext, Classical.choice, Quot.sound}; haec_ess, NE_iff, god_exists_succ, noN_S5, noA2_S5 axiom-free; 1134 search certificates re-evaluated by a direct evaluator per varying mode; table identical for possibly-empty,

nonempty and every-individual-somewhere domains; const mode equals K13 pattern; no countermodel contradicts a Lean cell; 288-cell pattern asserted, no OPEN, sha256-pinned — pass

Prior art. Partial prior art.

Claim K18

Varying-domain rigid Scott, 64 subsets of A1a,A1b,A2..A5 × 9 classes: A1b is never needed; $\exists^E G$ iff {A1a,A2,A3,A5} on T,B,S4,S5 or +A4 on KB; $\Box\exists^E G$ iff {A1a,A2,A3,A5} on T,B,S4,S5,KD45 or +A4 on any frame.

Status: derived ůa author check only.

Verified.

- `checks/K18-c009_varying_rigid_split.py`: lean –stdin exit 0, no sorry, no axiom decls; god_nonempty, serial, T_god, T_box, eucl_box, A4_box, sym_A4_god (all with A1a only) within {propext, Classical.choice, Quot.sound}; haec_ess, NE_iff, god_exists_succ, A4_god_back, godel_no_world, kd45_A4, kb_noA4 axiom-free; 4274 search certificates re-evaluated by a direct evaluator; 1152-cell pattern asserted with no OPEN cell; table identical for nonempty domains; constant mode matches K7/K13 split pattern; sha256 70aa8973... — pass
- `checks/K18-c009_varying_rigid_split.py`: lean –stdin exit 0, no sorry, no axiom decls; 14 theorems within standard axioms; 4274 search certificates re-evaluated; 1152-cell pattern asserted with no OPEN cell; sha256 70aa8973... — pass

Prior art. Partial prior art.

Claim K21

Anderson G_A, full properties, 64 subsets of A1a,A1b,A2,H,A4,A5 × 9 classes × constant/varying domains: collapse iff {A1a,A1b,A2,H} on B,S5, or +A4 on KB; $\exists G_A$ on KD45 needs A4 or A1b and constant domains; otherwise as K9/K15.

Status: derived ůa author check only.

Verified.

- `checks/K21-c009_anderson_a1b.py`: lean –stdin exit 0, no sorry, no axiom decls; god_det, eucl_A1b_godlike, eucl_A4_godlike, refl_sym_collapse, sym_A4_collapse, sym_god, sym_box, eucl_box, A2H_A5, kb_cycle, kd45_A1b and 6 helpers within {propext, Classical.choice, Quot.sound}; 16528 bound models + 5 named models re-evaluated; no model contradicts

a Lean cell; 1152-cell pattern asserted, all consistent, no OPEN; sha256 08d76079... — pass

Prior art. Partial prior art.

Claim K22

Full-property Scott, constant or varying domains, 64 subsets of A1a,A1b,A2–A5 × 2 essences × 9 classes: A1b is never needed; $\exists G$ iff {A1a,A2,A3,A5} on B,KB,S5 (box also KD45); collapse iff {A1a,A2,A5} there; Gödel essence with {A1a,A2,A5} has no model.

Status: derived ů author check only.

Verified.

- `checks/K22-c009_scott_split.py`: lean –stdin exit 0, no sorry, no axiom decls; serial, T1, A125_sym_collapse, A1235_sym_god, A1235_euclid_box, godel1970_no_model (hypotheses A1a only, E free) within {propext, Classical.choice, Quot.sound}; haec_ess, entails_top axiom-free; K3 data sha256 91c8ed5b... pinned; 3116 model outcomes re-evaluated (class, split axioms, refuted outcome) incl. the all-positive model; 864-cell pattern asserted, A1b-drop invariance asserted, table sha256 eb8f629f... pinned — pass

Prior art. Partial prior art.

Claim K24

Constant-domain full-property simplified AllMeet slice (216 cells): every cell is consistent and permits noncollapse; $\exists G$ and $\Box\exists G$ hold exactly for all three axioms with local/either closure. Every other cell has a finite countermodel.

Status: derived ů author check only.

Verified.

- `checks/K24-allmeet_classification.py`: Self-contained Lean existence/box proofs plus regeneration of SHA256-pinned 216-cell AllMeet table; every finite certificate re-evaluated; arbitrary meets checked exactly by top and binary intersections on finite powersets; no open cells. — pass

Prior art. Partial prior art.

Claim K28

Varying-domain full-property simplified slices: AllMeet has exactly K24s 216-cell outcomes; binary closure has 216 consistent, noncollapsing cells refuting

$\exists^E G$ and $\Box\exists^E G$, even with A1w. Positive AllMeet cells require all axioms and local/either closure.

Status: derived ů author check only.

Verified.

- `checks/K28-varying_meets.py`: Lean core with arbitrary E proves AllMeet PG/existence/box under A1prime and guarded local closure, plus either-to-local; guarded total-E tail model refutes existence/box and inherited proofs certify binary closure/A1w/noncollapse. Regenerates SHA256-pinned 216-cell AllMeet table and reevaluates every finite certificate with explicit E; enumerates 216 binary rows by weakening the Lean infinite model; no sorry or declared axioms. — pass

Prior art. Partial prior art.

Discussion

The map is a refinement of work that is largely known, not a new argument. Its value is completeness: for each variant, every subset of the axioms and every listed frame class is decided with a certificate, so the minimal sets and the frame boundaries can be read off rather than inferred from separate studies. Several entries agree with results already established with other provers [8, 4, 9], which we take as a check on our encoding rather than as findings.

The map covers only the stated choices: a shallow embedding of higher-order modal logic, Scott's and Gödel's definitions of essence, the listed variants, nine frame classes and two kinds of domain. Other definitions of essence, other readings of positivity (for example world-relative and rigid mixtures), Gödel's conjunction axiom in all slices [3], and logics beyond these frame classes are outside it. A cell decided here can therefore change under a different encoding, and claims of minimality hold relative to the listed axioms.

What a reader can take away is where the strength of each variant sits: in Scott's version the weight lies on A1a, A2, A3 and A5 together with symmetry, while A1b and A4 are idle; in Anderson's version the dropped complement direction is exactly the switch for collapse; and in the rigid and simplified readings, existence holds without collapse. Natural next steps are the open cells the run left (mixed conjunction principles in some slices) and other definitions of essence.

Provenance. Run `20261018-1933`, claims K2, K7, K9, K13, K15, K16, K18, K21, K22, K24, K28, run-folder commit `338871c09385f72bc0f657c9a639ef6a3c825566`, published 2026-10-19.

6 Settling informal claims about modal agents in Chicken

Setting

Modal agents play one-shot games by asking what can be proved about each other, written with the provability box $\Box$ of the logic GL [1]. Two informal posts made claims about specific agents that were never checked. "Modal Chicken" [17] introduced CarefulBot and AstuteBot for the game of Chicken, and its comments proposed graded variants: $C_n = \neg\Box(\Box^n\bot \vee p)$ ("swerve unless the opponent provably swerves", with proof strength n) and BoldBots $B_n = \Box(\Box^n\bot \vee \neg p)$, the name a commenter gave to the post's original CarefulBot (B_0); here p means "the opponent swerves against me". "Yet More Modal Combat" [18] proposed, for the prisoner's dilemma, a bot that defects if a weak system proves the opponent cooperates and otherwise acts like FairBot with a strong system; in graded form $H_{a,b} = \neg P_a(p) \wedge P_b(p)$ with $P_a(z) = \Box(\Box^a\bot \vee z)$, $a < b$.

Result

- **Graded CarefulBots and BoldBots, for all grades:** C_n swerves against C_m exactly when $n \leq m$, while B_n swerves against B_m exactly when $n > m$. A CarefulBot against a BoldBot always goes straight while the BoldBot swerves, so the pair always ends at payoffs $(4, 1)$, whatever the grades.
- **The named agents:** a certified table of all 64 pairings of SwerveBot, NerveBot, the original CarefulBot B_0, C_0, AstuteBot as displayed in the post, the FairBots $P_0(p) = \Box p$ and $P_1(p)$, and Hobson's bot $H_{0,1}$ (which cooperates with itself but ends in mutual defection with both FairBots). The original CarefulBot crashes against itself, as a comment pointed out; the displayed AstuteBot swerves against itself, as the post claimed.
- **Hobson's bot, for all grades:** $H_{a,b}$ and $H_{c,d}$ end in mutual cooperation exactly when their weak grades agree ($a = c$), and otherwise in mutual defection. Against any FairBot of the form $P_e(p)$ it ends in mutual defection, for every e.

Why it matters

Informal claims about small agents are easy to make and easy to get wrong; two of the claims in these posts were already disputed in their comment threads. These results settle them for every grade at once. They also show that Hobson's bot, proposed as a bot that cooperates with FairBots, does not cooperate with any graded FairBot of this form.

Proved vs checked

The graded laws (CarefulBots, BoldBots, the mixed pair, Hobson's bot) are proved for all indices in Lean 4, in the height semantics of closed GL fixed points (see [11]; [24]). The 64-cell table is certified by GL fixed-point computation.

This result extends the informal claims about CarefulBot, AstuteBot and graded CarefulBots and BoldBots in "Modal Chicken" and its comments (Gram_Stone 2015); Hobson's two-system bot and the counterexample in its comments (Hobson 2021). Our prior-art search (2026-10-20) covered:

- Mennen, "Modal combat for games other than the prisoner's dilemma" (LessWrong / Alignment Forum, 2017): framework for modal agents in games with any number of players and actions (player types, ranks, opponent tests naming a target player, an agent to test against and an action); existence and Pareto theorems; no agent families with index laws, no enumeration.
- Gram_Stone, "Modal Chicken" (LessWrong, 2015) and its comments (Alex-Mennen, SilentCal): SwerveBot, NerveBot, CarefulBot, AstuteBot; informal claims about graded CarefulBots and BoldBots.
- Hobson, "Yet More Modal Combat" (LessWrong / Alignment Forum, 2021): a two-system PD bot; generalisation to Chicken and more players left open.
- Critch, Dennis & Russell, "Cooperative and uncooperative institution designs: surprises and problems in open-source game theory" (arXiv 2208.07006): two-player bounded proof-search agents.
- Todorov, ten Napel & Müller, "Parametric Open Source Games" (arXiv 2606.27068): non-modal parametric agents; Chicken and Stag Hunt thresholds.
- Barasz et al. 2014 (arXiv 1401.5577); LaVictoire et al. 2014; Critch, parametric bounded Löb (arXiv 1602.04184); Oesterheld 2022 (arXiv 2211.05057); LessWrong / Alignment Forum wiki "Modal combat"; web searches for modal or provability-based agents in Chicken, Stag Hunt and multiplayer program equilibrium.

If you know of related work we missed, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was partly checked by scripts that run Lean proofs (claims K5, K11, K12) and partly checked by script (claims K6); the proof and check files are included so anyone can re-run them.

Claim K5

For all $n, m \geq 0$, $C_n = \neg P_n(p)$ swerves against C_m iff $n \leq m$, while $B_n = P_n(\neg p)$ swerves against B_m iff $n > m$, where $P_n(a) = \Box(\Box^n \bot \vee a)$. Exact height laws are Lean-proved.

Status: derived ů author check only.

Verified.

- `checks/K5-informal_tables.py`: 50 graded pair outcomes for indices 0..4, each with both simultaneous GL fixed-point equations certified. — pass
- `checks/K5-graded_families.py`: Lean core proves exact height laws for every n,m,h and every solution, plus boxed-formula bridge; no sorryAx; false tie law rejected; 20480 finite recurrence cells agree. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K6

The 64-cell GL table for $S, N, B_0, C_0, A, H, F_0, F_1$ is certified: original CarefulBot B_0 self-crashes; displayed AstuteBot A self-swerves; $H = \neg\Box p \wedge \Box(\Box\bot \vee p)$ self-cooperates but mutually defects with F_0, F_1.

Status: derived ů author check only.

Verified.

- `checks/K5-informal_tables.py`: All 64 stated outcomes, both GL fixed-point equations per cell, rejection of flipped candidates, and closed-form tail evaluation. — pass
- `checks/K5-informal_tables.py`: 64 outcomes and both simultaneous GL equations per pair, flipped controls rejected; no calculation changed. — pass

Prior art. Partial prior art.

Claim K11

For all $n, m \geq 0$, $C_n = \neg P_n(p)$ goes straight and $B_m = P_m(\neg p)$ swerves against it at every height, where $P_k(a) = \Box(\Box^k\bot \vee a)$. The mixed pair always yields Chicken payoffs $(4, 1)$, independent of grades (Lean).

Status: derived ů author check only.

Verified.

- `checks/K11-mixed_graded.py`: Lean proves mixed_all for all natural indices/heights and every solution, with no axioms; boxed-formula bridge; 25 GLS cells and 10240 recurrence cells pass; false conclusion rejected. — pass

Prior art. Partial prior art.

Claim K12

For $H_{a,b} = \neg P_a(p) \wedge P_b(p)$, $P_a(z) = \Box(\Box^a \bot \vee z)$, all $a < b, c < d$ give eventual mutual action 1 iff $a = c$, else mutual action 2. Every pairing with $P_e(p)$ eventually gives mutual action 2, for all e (Lean).

Status: derived ů author check only.

Verified.

- `checks/K12-hobson_grades.py`: 100 generalized Hobson pair cells plus 50 graded FairBot cells; both fixed-point equations GL-proved per cell, flipped candidates rejected. — pass
- `checks/K12-hobson_all_indices.py`: Copied another researcher proof: Lean hob_eq, hob_tail, hob_fair for all indices, boxed bridge, no sorryAx; two false variants rejected; 7098 recurrence cells. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Discussion

The results depend on reading each informal definition as a specific modal formula; where a post was ambiguous we state the formula we used, and a different reading could change the outcome. Hobson's proposal allows other pairs of systems than the graded ones here, so the negative result covers this graded family, not every possible instantiation.

Provenance. Run `20261019-1400`, claims K5, K6, K11, K12, run-folder commit `24c2a64bf8b8cb8be184b4d449ce11068d8270ea`, published 2026-10-20.

7 Modal agents in the game of Chicken

Setting

Two programs play a one-shot game and each can read the other's source code. Following Barasz et al. [1], a *modal agent* decides by asking what can be proved about the opponent, written with the provability box $\Box$ of the logic GL. Earlier findings on this site study the prisoner's dilemma; here the game is mostly **Chicken**: both swerve 3/3, swerve against straight 1/4, both go straight (crash) 0/0. p means "the opponent swerves against me", q "the opponent swerves against NerveBot", the bot that never swerves. SwerveBot always swerves; FairBot is $\Box p$, graded FairBots are $\Box^j p$. The census covers every agent built from p, q, $\bot$, $\neg$, $\wedge$, $\vee$ and $\Box$ (atoms under a box) up to size 10, 7561 behaviourally

distinct agents. Mennen [21] set up modal agents for general games; this finding works out Chicken concretely.

Result

- **Equilibria never crash.** Among the census agents there are 517 symmetric program equilibria, all with mutual swerving, and no equilibrium at all involves a crash. Every asymmetric equilibrium pairs one of the 701 agents that never swerve against anyone in the census with an agent that swerves against it.
- **The smallest sensible Chicken agents** (swerve against themselves and against NerveBot, go straight against SwerveBot) have size 3: exactly $\neg\Box p$ and $\neg\Box q$. The first never crashes against any census agent; the second crashes against 755 of them.
- **Safe and fair.** Exactly 4 census agents never crash against a census agent, go straight against SwerveBot and swerve together with FairBot, and all 4 never crash against any modal opponent at all. The smallest, of size 9, is the $k = 2$ member of a family $Q_k = \Box^k\bot \vee \neg\Box(p \wedge q)$ with exact laws for every k: Q_k never crashes, beats SwerveBot, swerves against itself, swerves together with $\Box^j p$ exactly when $j < k$ and with $\Box(\Box^j\bot \vee p)$ exactly when $j + 2 \leq k$, and goes straight against Q_m for $m > k$.
- **A limit for every two-action game.** If an agent of modal depth d exploits the always-cooperating bot, then graded FairBots $\Box^j p$ with $j \geq d$ end up playing action 2 against it, and the two never settle on mutual action 1. The Q_k family shows the bound is tight.
- **Tournaments and evolution.** In a round robin the top Chicken agent is $\neg\Box p \wedge \Box(\neg p \vee \Box p)$, which crashes against itself. Under replicator dynamics the surviving agents are the same 1116 classes in the prisoner's dilemma, Chicken and Stag Hunt, and they form a maximal set of mutual cooperators containing FairBot.

Why it matters

Modal agents have been studied almost only in the prisoner's dilemma. Chicken punishes mutual defection (a crash) instead of rewarding it, so the question changes from "can I avoid being exploited?" to "can I avoid crashing without being a pushover?". The census answers it concretely: equilibria never crash, safe agents exist and form a family with exact thresholds, and a general depth bound limits how such agents can coexist with graded FairBots.

Proved vs checked

The laws for Q_k and the depth bound are proved for all indices in Lean 4, in the height semantics of closed GL fixed points (see [11]; the link to provability

is Solovay's theorem [24]). The census results (equilibria, smallest agents, the four safe agents, round robin, replicator dynamics) hold for all agents up to size 10, computed exactly and re-checked on samples; the replicator results are numerical (2000 steps from the uniform start).

We believe this result is new. Our prior-art search (2026-10-20) covered:

- Mennen, "Modal combat for games other than the prisoner's dilemma" (LessWrong / Alignment Forum, 2017): framework for modal agents in games with any number of players and actions (player types, ranks, opponent tests naming a target player, an agent to test against and an action); existence and Pareto theorems; no agent families with index laws, no enumeration.
- Gram_Stone, "Modal Chicken" (LessWrong, 2015) and its comments (Alex-Mennen, SilentCal): SwerveBot, NerveBot, CarefulBot, AstuteBot; informal claims about graded CarefulBots and BoldBots.
- Hobson, "Yet More Modal Combat" (LessWrong / Alignment Forum, 2021): a two-system PD bot; generalisation to Chicken and more players left open.
- Critch, Dennis & Russell, "Cooperative and uncooperative institution designs: surprises and problems in open-source game theory" (arXiv 2208.07006): two-player bounded proof-search agents.
- Todorov, ten Napel & Müller, "Parametric Open Source Games" (arXiv 2606.27068): non-modal parametric agents; Chicken and Stag Hunt thresholds.
- Barasz et al. 2014 (arXiv 1401.5577); LaVictoire et al. 2014; Critch, parametric bounded Löb (arXiv 1602.04184); Oesterheld 2022 (arXiv 2211.05057); LessWrong / Alignment Forum wiki "Modal combat"; web searches for modal or provability-based agents in Chicken, Stag Hunt and multiplayer program equilibrium.

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was partly checked by scripts that run Lean proofs (claims K7, K8, K10) and partly checked by script (claims K1, K2, K3, K4); the proof and check files are included so anyone can re-run them.

Claim K1

`In L up to size 10 (7561 classes; Chicken 3/3, 1/4, 0/0), census program equilibria: 517 symmetric, all mutual swerve; none with a crash; every swerve/straight one pairs one of the 701 agents that never swerve against any census agent with an agent swerving against it.`

Status: derived ů author check only.

Verified.

- `checks/K1-c046_games10.py`: (1) class counts; (2) independent re-solve + 13 GLS certificates; (3) Chicken equilibrium counts 517 symmetric all CC, 262741 CC, 2138719 CD/DC, and CD-equilibrium iff straight player never swerves (701 classes) and the other swerves vs it — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K2

In L up to size 10, round robin incl. self: Chicken (3,1,4,0) max 17666 only by $\neg\Box p \wedge \Box(\neg p \vee \Box p)$, which crashes vs itself (SwerveBot 16479, FairBot 14303); Stag Hunt (4,0,3,2) max 23411 only by $\Box(p \vee \Box^3\neg(p \vee q))$, which hunts stag vs HareBot (FairBot 21719).

Status: derived ů author check only.

Verified.

- `checks/K1-c046_games10.py`: (5) unique RR maxima 17666 (Chicken) and 23411 (Stag Hunt) with stated agents of size 10 and stated scores of CB, DB, FB; leader cells vs self/DB/CB/FB GLS-certified in (2) — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K3

In L up to size 10, discrete replicator dynamics from uniform start (2000 steps) leave exactly the same 1116 observational classes (weight $> 10^{-6}$) in PD, Chicken and Stag Hunt; they form a maximal set of mutual cooperators containing FairBot.

Status: derived ů author check only.

Verified.

- `checks/K3-c046_replicator10.py`: 3402 observational classes; in each game survivors >1e-6 are the same 1116 (2158 chain classes), others <1e-12, mean payoff R within 1e-9; survivors pairwise CC (exact) and maximal (exact); FB in S, DB not — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K4

In L up to size 10, the smallest agents that swerve vs themselves and vs NerveBot but go straight vs SwerveBot have size 3: exactly $\neg\Box p$ and $\neg\Box q$; $\neg\Box p$ crashes against no census agent, $\neg\Box q$ against 755.

Status: derived ů author check only.

Verified.

- `checks/K1-c046_games10.py`: (6) property set minimum size 3 = {~[]p, ~[]q}; ~[]p no crash vs any of 7561; ~[]q 755 crashes; no census agent anti-coordinates with all others; ~[]p cells vs self/F/~F/[]p GLS-certified in (2) — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K7

For all $k \geq 0$, $Q_k = \Box^k \bot \vee \neg\Box(p \wedge q)$ never crashes vs any opponent, beats SwerveBot, self-swerves; it mutually swerves with $\Box^j p$ ($j \geq 1$) iff $j < k$ and with $\Box(\Box^j \bot \vee p)$ iff $j+2 \leq k$, else is exploited; vs Q_m, $m > k$, Q_k goes straight. Lean.

Status: derived ů author check only.

Verified.

- `checks/K7-c046_chicken_q.py`: (5) Lean 4 core theorems q_safe, q_sb, q_nb, q_fj, q_gj, q_db, q_qm_lt, q_qq, q_bn for all indices and any solving traces, no sorryAx, negative control (j+1<=k for G_j) rejected; (4) height tables for k,j,m,n<=8 agree; (3) GLS certificates for Q_2 cells — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K8

In L up to size 10 (Chicken), exactly 4 agents crash against no census agent, go straight vs SwerveBot and mutually swerve with FairBot; all 4 never crash vs any modal opponent; the unique smallest (size 9) is $\Box^2 \bot \vee \neg\Box(p \wedge q)$.

Status: derived ů author check only.

Verified.

- `checks/K7-c046_chicken_q.py`: (1) 7561 classes, 808 census-safe, 695 SU, SU subset of safe; (2) safe & straight vs CB & CC with FB: exactly 4, sizes 9,10,10,10, all SU, unique of size<=9 is ([][]F|~[](p&q)); Q_0, Q_1 fail only the FB clause; (3) 9 GLS certificates — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K10

For every guarded agent A of modal depth d playing action 2 vs the constant action-1 bot: $\Box^j p$ ($j \geq d$) plays 2 vs A from height $d + j$, and $\Box(\Box^j \bot \vee p)$ ($j \geq d - 1$) from height $j + 2$; never mutual action 1 (any 2-action game; Lean). Both tight via Q_k.

Status: derived ů author check only.

Verified.

- `checks/K10-c046_depth_bound.py`: (1) Lean 4 core theorem depth_bound over an inductive formula type (locality + stabilisation lemmas), no sorryAx, two weakened variants rejected; (2) size<=10 census: 7561 classes, constant vs SwerveBot from height md, 3102 straight vs SwerveBot, 12407 (agent,j>=md) pairs with F_j straight from md+j, 92 tight classes, smallest []([]F|~q); (3) explicit height tables — pass (Lean-proved, no sorryAx)
- `checks/K10-c046_depth_bound_g.py`: (1) Lean 4 core: depth_bound, straight_at, depth_bound_G over inductive Fm, no sorryAx, four weakened variants rejected; (2) size<=10 census: 3102 straight-vs-SB classes, 12407 F_j and 12407 G_j pairs straight from md+j resp. j+2, largest CC index vs G_j = vs F_j minus 1, 178 CC, 92 tight (same sets), smallest []([]F|~q); (3) explicit height tables — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Discussion

The census stops at size 10, and round-robin winners in particular can change as larger agents are added. The language has a single probe (q, how the opponent treats NerveBot); richer probes would give a larger space. The replicator results describe this population, not modal agents in general. A natural next step is Stag Hunt, where the round-robin leader found here is unusual, and a search for threshold families beyond FairBot-like agents there.

Provenance. Run `20261019-1400`, claims K1, K2, K3, K4, K7, K8, K10, run-folder commit `24c2a64bf8b8cb8be184b4d449ce11068d8270ea`, published 2026-10-20.

8 Three-player modal agents in a public-goods game

Setting

Modal agents decide what to do by asking what can be proved about the other players, written with the provability box $\Box$ of the logic GL [1]. Mennen [21]

set up a framework for more than two players; here we study three players concretely. The game is a public-goods game: each player contributes ($a_i = 1$) or not, and earns $u_i = 2\sum_j a_j - 3a_i$, so contributing costs 1 net but gives each other player 2. An agent sees p_1, p_2 ("opponent j contributes in this game") and probes q_1, q_2 ("opponent j contributes when I am replaced by a defector"). Behaviour is read off in the long run (tail semantics). An agent is *safe* if it never earns less than either opponent, whoever they are.

Result

- **The three-player FairBot is safe.** $F = \Box(p_1 \wedge p_2)$ (contribute iff it is provable that both others contribute) never earns less than either opponent against any two modal opponents, contributes with two copies of itself and defects against a defector.
- **It is the smallest.** No agent smaller than F (size 4) both cooperates with copies of itself and is safe.
- **All small safe cooperators are FairBot-like.** Exactly 66 agents up to size 6 cooperate with themselves and are safe; none of them uses the probes, and for each GL proves that it contributes only if $\Box^3(p_1 \wedge p_2)$. Every other agent of that size fails against some pair of opponents of size at most 3.
- **A prudent three-player agent needs the probe.** $E = \Box((p_1 \wedge p_2) \wedge (\Box\bot \vee \neg q_1))$ (size 10) cooperates with itself, is safe, and defects against two unconditional contributors; no agent of size at most 7 has all three properties.
- **The definition of the probe matters.** For five basic three-player bots we certify all 125 triples. Changing the probe from "only I am replaced by a defector" to "everyone except the queried opponent is replaced" changes the outcome of 9 of the 125 triples.

Why it matters

Program equilibrium with modal agents has been worked out almost only for two players. In a three-player public-goods game the natural FairBot generalisation stays safe, is the smallest safe cooperator, and is essentially the only kind up to size 6. Exploiting unconditional contributors requires the probe, as with PrudentBot in two-player games, and the exact definition of the probe is a real modelling choice, since outcomes change with it.

Proved vs checked

The safety of F against all modal opponents is proved in Lean 4 under tail semantics. The minimality, the 66-agent classification, the properties of E and

the 125-triple tables are exact computations over all agents up to the stated sizes in the stated language, with GL certificates.

We believe this result is new. Our prior-art search (2026-10-20) covered:

- Mennen, "Modal combat for games other than the prisoner's dilemma" (LessWrong / Alignment Forum, 2017): framework for modal agents in games with any number of players and actions (player types, ranks, opponent tests naming a target player, an agent to test against and an action); existence and Pareto theorems; no agent families with index laws, no enumeration.
- Gram_Stone, "Modal Chicken" (LessWrong, 2015) and its comments (Alex-Mennen, SilentCal): SwerveBot, NerveBot, CarefulBot, AstuteBot; informal claims about graded CarefulBots and BoldBots.
- Hobson, "Yet More Modal Combat" (LessWrong / Alignment Forum, 2021): a two-system PD bot; generalisation to Chicken and more players left open.
- Critch, Dennis & Russell, "Cooperative and uncooperative institution designs: surprises and problems in open-source game theory" (arXiv 2208.07006): two-player bounded proof-search agents.
- Todorov, ten Napel & Müller, "Parametric Open Source Games" (arXiv 2606.27068): non-modal parametric agents; Chicken and Stag Hunt thresholds.
- Barasz et al. 2014 (arXiv 1401.5577); LaVictoire et al. 2014; Critch, parametric bounded Löb (arXiv 1602.04184); Oesterheld 2022 (arXiv 2211.05057); LessWrong / Alignment Forum wiki "Modal combat"; web searches for modal or provability-based agents in Chicken, Stag Hunt and multiplayer program equilibrium.

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result extends Mennen's framework for multiplayer modal agents, whose opponent tests fix one definition of probe atoms (Mennen 2017). Our prior-art search (2026-10-20) covered:

- Mennen, "Modal combat for games other than the prisoner's dilemma" (LessWrong / Alignment Forum, 2017): framework for modal agents in games with any number of players and actions (player types, ranks, opponent tests naming a target player, an agent to test against and an action); existence and Pareto theorems; no agent families with index laws, no enumeration.
- Gram_Stone, "Modal Chicken" (LessWrong, 2015) and its comments (Alex-Mennen, SilentCal): SwerveBot, NerveBot, CarefulBot, AstuteBot; informal claims about graded CarefulBots and BoldBots.

- Hobson, "Yet More Modal Combat" (LessWrong / Alignment Forum, 2021): a two-system PD bot; generalisation to Chicken and more players left open.
- Critch, Dennis & Russell, "Cooperative and uncooperative institution designs: surprises and problems in open-source game theory" (arXiv 2208.07006): two-player bounded proof-search agents.
- Todorov, ten Napel & Müller, "Parametric Open Source Games" (arXiv 2606.27068): non-modal parametric agents; Chicken and Stag Hunt thresholds.
- Barasz et al. 2014 (arXiv 1401.5577); LaVictoire et al. 2014; Critch, parametric bounded Löb (arXiv 1602.04184); Oesterheld 2022 (arXiv 2211.05057); LessWrong / Alignment Forum wiki "Modal combat"; web searches for modal or provability-based agents in Chicken, Stag Hunt and multiplayer program equilibrium.

If you know of related work we missed, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was partly checked by scripts that run Lean proofs (claims K14, K16, K18, K19) and partly checked by script (claims K13, K21); the proof and check files are included so anyone can re-run them.

Claim K13

For five three-player bots $D, C, F = \Box(p_1 \wedge p_2), O = \Box(p_1 \vee p_2), Q = \Box(q_1 \wedge q_2)$, 125 triples are GL-certified under evaluator-replacement probes: $FFF = 111$, $FFD = 000$, $QQQ = 000$; exactly D, F, Q never earn less than either opponent within this five-bot set.

Status: derived ů author check only.

Verified.

- `checks/K13-three_player.py`: 125 triples, all 3000 simultaneous replacement-subset equations GL-certified; 375 flipped-candidate controls; headline cells and finite payoff/self-play classifications. — pass
- `checks/K13-probe_semantics_alternative.py`: Re-certifies the original 125 triples and alternative 125 triples; 6000 equations and 750 flip controls; exactly nine stated differences. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K14

In the three-player public-goods game $u_i = 2\sum_j a_j - 3a_i$, $F = \Box(p_1 \wedge p_2)$ never earns less than either opponent against any two modal opponents (Lean, tail semantics), while cooperating with two copies and defecting against a defector.

Status: derived ů author check only.

Verified.

- `checks/K14-c046_xreview_k13.py`: (4) Lean 4 core fair3_tail (any opponent traces, X->Y&Z) and fair3_payoff (decided over 8 tails), no sorryAx, false variant rejected; (2) FFF=111, FFD=000, QQQ=000, FCD=010, QBC=101 GL-certified on all 24 replacement equations; (1) 125 K13 triples re-solved independently — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K16

In the guarded three-player language with node-count size and evaluator-replacement probes, size 4 is minimal for self-cooperation plus never earning less than either opponent in $u_i = 2\sum_j a_j - 3a_i$: $F = \Box(p_1 \wedge p_2)$ attains it (tail semantics).

Status: derived ů author check only.

Verified.

- `checks/K16-three_player_minimal.py`: Exhaustive 25 raw guarded trees through size 3; 15625 augmented triples, 375000 GL equations, 46875 flip controls; each smaller self-cooperator has a disadvantage witness; size-4 FairBot self-cooperates and passes finite safety. — pass
- `checks/K16-three_player_minimal.py`: 25 exhaustive raw trees through size 3; 15625 triples / 375000 GL equations / 46875 flip controls; smaller self-cooperators have disadvantage witnesses; embedded K14 Lean fair3_tail and fair3_payoff prove universal upper bound, no sorryAx, false variant rejected. — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K18

In the guarded three-player language (evaluator-replacement probes, $u_i = 2\sum_j a_j - 3a_i$), exactly 66 trees of size ≤ 6 self-cooperate and never earn less than either opponent; all are q-free and GL proves $A \to \Box^3(p_1 \wedge p_2)$; others fail vs size-$\leq 3$ pairs.

Status: derived ů author check only.

Verified.

- `checks/K18-c046_safe6.py`: (1) 1,6,18,112,596,3756 guarded trees; (2) self-cooperators 11,41,263,1448; survivors 0,0,0,2,8,56, all others beaten by size<=3 pairs (constants except 8 named trees); (3) survivors q-free, GL proves A->[][][](p1&p2), control rejected; (4) 3438 triples GL-certified, 24 equations each, flips rejected; (5) Lean lag_tail, fair3_payoff, no sorryAx, false variant rejected — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K19

In K18's three-player language, $E = \Box((p_1 \wedge p_2) \wedge (\Box\bot \vee \neg q_1))$ (size 10) self-cooperates, defects against two constant contributors (payoffs 4,1,1) and never earns less than either opponent; no tree of size ≤ 7 has all three properties.

Status: derived ů author check only.

Verified.

- `checks/K19-c046_prudent3.py`: (1) 9 E-cells (EEE=111, ECC=011, ECD=010, EDD=000, E FB FB=111, E E FB=111, E C FB=010, E D FB=000, EEC=111) GL-certified, 24 equations each, flips rejected; (2) GL proves E->[](p1&p2), control rejected; (3) sizes<=7: 1,6,18,112,596,3756,23636 trees, self-coop & defect vs (C,C) 0,0,4,4,68,220,1808, each beaten by a size<=3 pair; (5) Lean lag_tail, fair3_payoff, no sorryAx, false variant rejected — pass (Lean-proved, no sorryAx)

- `checks/K19-c046_xreview_k20.py`: K20 cells incl. RRR=000 (q1->p1) GL-certified on 24 equations, flips rejected; GL proves A<->E with 8 mixed triples 111; GL rejects A<->R (probe essential) — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K21

For K13s five bots, replacing the evaluator alone versus replacing all players other than the queried opponent changes exactly 9 of 125 triples: permutations of (Q, C, O) and (Q, O, O) change from 111 to only Q defecting (GL-certified).

Status: derived ů author check only.

Verified.

- `checks/K13-probe_semantics_alternative.py`: 250 triples, 6000 GL equations, 750 flips; exactly the specified nine differences, QOC 111 versus 011; all probe-free triples unchanged. — pass

Prior art. Partial prior art.

Discussion

The results use one language (two "contributes" atoms and two probes), one payoff function and tail semantics; other probe definitions change some outcomes, as shown. The classifications stop at sizes 6 to 10. Open questions are the minimum size of a safe self-cooperator that needs the probe (between 8 and 10) and whether these patterns extend to more players or other multiplayer games.

Provenance. Run `20261019-1400`, claims K13, K14, K16, K18, K19, K21, run-folder commit `24c2a64bf8b8cb8be184b4d449ce11068d8270ea`, published 2026-10-20.

9 Blending self-defecting modal agents with FairBot: exact laws

Setting

Two programs play a one-shot prisoner's dilemma, and each can read the other's source code. Following Barasz et al. [1], a *modal agent* decides by asking what can be proved about the opponent, written with the provability box $\Box$ of the logic GL: p means "the opponent cooperates with me" and q "the opponent cooperates with DefectBot". FairBot is $\Box p$, graded FairBots are $F_k = \Box^k p$. An agent is *unexploitable* if no modal opponent ever ends up defecting while the agent cooperates. Outcomes are computed in the height semantics: a game is a sequence of moves at heights $0, 1, 2, \ldots$, and its result is the eventual move.

Earlier findings on this site describe infinite families of unexploitable agents that cooperate with FairBot and CooperateBot but defect against their own copy: the gate families $V_{n,r} = \Box(\Box^n \bot \vee (p \wedge (q \vee \neg\Box^r q)))$, with $W_n = V_{n,1}$, and $J_{a,b,j} = \Box^{a+1}(\Box^b p \wedge (q \vee \Box(\Box^j \bot \vee \neg q)))$. Here we ask what happens when such an agent is *blended* with another agent X: a formula σ_S without atoms picks out a set S of heights (finite, or all but finitely many), and the blend plays like X at the heights in S and like the family member elsewhere. X is FairBot, a graded FairBot, DefectBot or CooperateBot.

Result

- **FairBot-type blends.** For a graded-FairBot blend with finitely many selected heights: - a W_n blend cooperates with itself exactly when S meets $\{k, \ldots, n-1\}$; - a $J_{a,b,j}$ blend exactly when $k \le j$ and S contains all of $\{j, \ldots, a+b\}$; - for $V_{n,r}$ the condition is a window determined by the blend's DefectBot trace, and it is not monotone: for $V_{3,2}$, selecting height

2 makes the blend cooperate with itself, while selecting heights 2 and 3 does not.

All these blends stay unexploitable and keep cooperating with FairBot and CooperateBot.

- **DefectBot blends.** Playing DefectBot at even one height rules out mutual cooperation with FairBot and with every F_k, because an agent ends in mutual cooperation with F_k exactly when it cooperates at every height of that game. A DefectBot blend of W_n still cooperates with itself exactly when S is a non-empty subset of $\{0, \ldots, n-1\}$; for $J_{a,b,j}$ never.
- **CooperateBot blends** cooperate with themselves only when CooperateBot is played at all but finitely many heights, and then DefectBot exploits them.
- **Not FairBot in disguise.** For finitely many selected heights, a single graded FairBot (F_{j+1} for J, $F_{\max(1,n-r+1)}$ for V) behaves differently against the blend than against FairBot or any $\Box^m p$. A blend selecting all but finitely many heights behaves exactly like FairBot only in the trivial case (FairBot at every height from 1 on); otherwise an explicit opponent tells them apart.

Why it matters

Unexploitable agents that defect against themselves seem to sit far from FairBot, which cooperates with itself. These laws say exactly how much FairBot behaviour, and at which heights, is needed to restore self-cooperation without losing safety, and they give an infinite supply of unexploitable self-cooperating agents that are provably not FairBot.

Proved vs checked

All laws are proved for all indices in Lean 4, in the height semantics of closed GL fixed points; the link to provability in arithmetic is Solovay's theorem [24], cited, not formalised (see also [11]). Finite grids of cases were also checked independently by GL fixed-point solving with certificates.

We believe this result is new. Our prior-art search (2026-10-21) covered:

- Barasz, LaVictoire, Christiano, Fallenstein, Herreshoff, Yudkowsky, "Robust Cooperation in the Prisoner's Dilemma: Program Equilibrium via Provability Logic" (arXiv 1401.5577): unexploitability (Section 2), rank and Theorem 4.10 (Section 4), WaitFairBot_K and the obstacle theorem (Section 5).
- LessWrong, "Is there only one FairBot?" (transhumanist_atom_understander, 29 Aug 2026) and comments: Zac Hatfield-Dodds shows the uniqueness conjecture holds up to modal depth 1 for rank-0 agents and fails from

depth 2 (Box Box p; FairBot over PA+1 = Box(p or Box False); Box^k p for k >= 3); PrudentBot separates Box Box p from FairBot; the author notes Box Box p is not provably unexploitable in PA. No enumeration or counts.

- LessWrong / Alignment Forum wiki "Modal combat": DefectBot, CooperateBot, FairBot, PrudentBot, CliqueBot, TrollBot; no uniqueness result, no enumeration.

- LaVictoire et al. 2014 (MIPC); comment thread on "Robust Cooperation in the Prisoner's Dilemma" (LessWrong, on instability of FairBot equilibria); Hobson, "Yet More Modal Combat" (2021); Critch, parametric bounded Löb (arXiv 1602.04184); Critch, Dennis & Russell (arXiv 2208.07006); MIRI provability repository (github.com/machine-intelligence/provability, a modal-agent evaluator, no published census).

- Out of scope by direction: probabilistic blends (Oesterheld arXiv 2211.05057; Cooper et al. arXiv 2412.14570).

- Web searches for deterministic blends, behavioural distance and classifications of modal agents (2026-10-20/21); none means not found in this search.

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result extends Barasz et al. (arXiv 1401.5577): FairBot, the height guards of WaitFairBot_K, and the evaluation of modal agents on linear GL frames; Boolos, The Logic of Provability (1993): semantics of GL; Our earlier published findings on W_n and the gate families. Our prior-art search (2026-10-21) covered:

- Barasz, LaVictoire, Christiano, Fallenstein, Herreshoff, Yudkowsky, "Robust Cooperation in the Prisoner's Dilemma: Program Equilibrium via Provability Logic" (arXiv 1401.5577): unexploitability (Section 2), rank and Theorem 4.10 (Section 4), WaitFairBot_K and the obstacle theorem (Section 5).

- LessWrong, "Is there only one FairBot?" (transhumanist_atom_understander, 29 Aug 2026) and comments: Zac Hatfield-Dodds shows the uniqueness conjecture holds up to modal depth 1 for rank-0 agents and fails from depth 2 (Box Box p; FairBot over PA+1 = Box(p or Box False); Box^k p for k >= 3); PrudentBot separates Box Box p from FairBot; the author notes Box Box p is not provably unexploitable in PA. No enumeration or counts.

- LessWrong / Alignment Forum wiki "Modal combat": DefectBot, CooperateBot, FairBot, PrudentBot, CliqueBot, TrollBot; no uniqueness result, no enumeration.

- LaVictoire et al. 2014 (MIPC); comment thread on "Robust Cooperation in the Prisoner's Dilemma" (LessWrong, on instability of FairBot equilibria); Hobson, "Yet More Modal Combat" (2021); Critch, parametric bounded Löb (arXiv 1602.04184); Critch, Dennis & Russell (arXiv 2208.07006); MIRI provability repository (github.com/machine-intelligence/provability, a modal-agent evaluator, no published census).

- Out of scope by direction: probabilistic blends (Oesterheld arXiv 2211.05057; Cooper et al. arXiv 2412.14570).

- Web searches for deterministic blends, behavioural distance and classifications of modal agents (2026-10-20/21); none means not found in this search.

- Boolos, The Logic of Provability (1993): GL semantics.

If you know of related work we missed, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was checked by scripts that run Lean proofs; the proof and check files are included so anyone can re-run them.

Claim K14

For $n \geq 2, k \geq 1$, an F_k height blend of W_n with finite S self-cooperates iff $S \cap \{k, \ldots, n-1\} \neq \varnothing$; all such blends remain unexploitable and cooperate with FairBot and CooperateBot.

Status: derived ů author check only.

Verified.

- `checks/K14-graded_self.py`: Six general Lean theorems: first-hole self-play law, DefectBot trace, first failure existence, early-hole criterion, FB and CB partners; SHA-pinned source, no sorryAx, three mutations rejected. — pass (Lean-proved, no sorryAx)

- `checks/K14-graded_grid.py`: 144 GL-certified cells for n2,3; k1,2,3; eight height selectors; self, FairBot, CooperateBot; wrong-candidate controls. — pass

- `checks/K14-blend_gate.py`: General graded-FairBot blend unexploitability and iterated-box semantics (K7). — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K6

For $n \geq 2$, a FairBot height blend of W_n self-cooperates iff the first false height t of its DefectBot trace satisfies $t < n$ or all $h > t$ belong to S; all such blends remain unexploitable.

Status: derived ů author check only.

Verified.

- `checks/K6-fb_self.py`: General Lean self-play iff law; first-hole and DefectBot trace semantics; first failure exists; early-hole criterion; all-height FB/CB cooperation; no sorryAx; three mutations rejected. — pass (Lean-proved, no sorryAx)
- `checks/K6-fb_grid.py`: All 48 GL-certified self/FB/CB cells for n=2,3 and eight selectors arbitrary on heights 0,1 and constant from 2; wrong-candidate controls. — pass
- `checks/K14-blend_gate.py`: General Lean unexploitability of all graded-FairBot blends, including FairBot. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K16

For an F_k height blend of $V_{n,r}$, let d be its DefectBot trace, t its first false height and $E = \{g \geq n : \neg d(g),\ g < t + r\}$. Self-play cooperates iff every height above every $g \in E$ is selected; for finite S, iff $E = \varnothing$.

Status: derived ů author check only.

Verified.

- `checks/K16-v_self.py`: 11 general Lean core theorems: obstruction law, first-hole window, finite-set law, singleton iff, DB trace, FB/CB cooperation, universal unexploitability; SHA-pinned, no sorryAx; 3 mutations rejected. — pass (Lean-proved, no sorryAx)
- `checks/K16-v_grid.py`: 486 GL-certified cells including all 480 grid cells for five (n,r) pairs, k1,2, 16 selectors vs self/FB/CB and six nonmonotonicity headline cells; wrong candidate controls. — pass
- `checks/K16-v_reestablished.py`: SHA-pinned full prior V-family script re-executed: nine Lean theorem checks, five rejected mutations, literal tables and 35 separated family members. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K19

For $1 \leq j \leq a+b$ and $k \geq 1$, an F_k height blend of $J_{a,b,j}$ with finite S self-cooperates iff $k \leq j$ and $\{j, \ldots, a+b\} \subseteq S$; all such blends are unexploitable and cooperate with FairBot and CooperateBot.

Status: derived ů author check only.

Verified.

- `checks/K19-j_self.py`: 9 SHA-pinned Lean core theorems: literal J normalization, general obstruction law, finite interval law, DB trace, universal unexploitability, FB/CB cooperation; no sorryAx; 3 rejected mutations. — pass (Lean-proved, no sorryAx)
- `checks/K19-j_grid.py`: 581 GL-certified cells: exhaustive six-triple/two-k/16-selector self-FB-CB grid and 5 headline cells including J111 selection1 versus2 versus1,2; F2 separator; wrong-candidate controls. — pass
- `checks/K19-j_reestablished.py`: Full SHA-pinned prior J-family script, including G and H special cases: literal tables, 70 distinct agents, formula-string cells, re-executed. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K9

A DefectBot height blend of W_n ($n \geq 2$) self-cooperates iff $\varnothing \neq S \subseteq \{0, \ldots, n-1\}$; a CooperateBot blend self-cooperates iff S is cofinite.

Status: derived ů author check only.

Verified.

- `checks/K9-other_self.py`: Lean core proofs of the general DB and CB self-play iff laws and both DefectBot-trace formulas; SHA-pinned source; no sorryAx; three mutations rejected. — pass (Lean-proved, no sorryAx)
- `checks/K9-other_grid.py`: All 32 GL-certified self-play cells for DB/CB blends, n=2,3, eight selectors arbitrary at 0,1 and constant from 2; wrong-candidate controls. — pass

Prior art. Partial prior art.

Claim K25

For $r \geq 1$, a DefectBot height blend of $V_{n,r}$ self-cooperates iff $S \subseteq \{0, \ldots, n-1\}$ and some $s \in S$ satisfies $s + r \leq n + 1$; all remain unexploitable. CooperateBot blends self-cooperate iff S is cofinite, when DefectBot exploits them.

Status: derived ů author check only.

Verified.

- `checks/K25-v_other.py`: Six SHA-pinned Lean core theorem groups: general DB iff, CB iff eventual selection, DB traces, DB safety, finite CB safety, cofinite CB exploitation; no sorryAx; three rejected mutations. — pass (Lean-proved, no sorryAx)
- `checks/K25-v_other_grid.py`: 896 GL-certified self/DB cells for 14 n,r pairs, 16 selectors, DB/CB branches; wrong-candidate controls. — pass
- `checks/K16-v_reestablished.py`: SHA-pinned prior V-family literal semantics and Lean results re-established as in K16. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K23

For $1 \leq j \leq a + b$, no DefectBot height blend of $J_{a,b,j}$ self-cooperates; a CooperateBot blend self-cooperates iff its selector is cofinite, and then DefectBot exploits it. Every DefectBot blend and every finite-selector CooperateBot blend is unexploitable.

Status: derived ů author check only.

Verified.

- `checks/K23-j_other.py`: Six SHA-pinned Lean core theorems for arbitrary indices/selectors: traces, no DB self-cooperation, CB self iff eventual selection, DB safety, finite CB safety, cofinite CB exploitation; no sorryAx; three rejected mutations. — pass (Lean-proved, no sorryAx)
- `checks/K23-j_other_grid.py`: All 384 self/DB cells GL-certified for six J triples, 16 selectors and DB/CB branches; wrong-candidate controls. — pass
- `checks/K19-j_self.py`: K19 literal J normalization into the height gate, SHA-pinned Lean proof. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K28

For every agent and $k \geq 1$: it ends CC with $F_k = \Box^k p$ iff it cooperates at every height of that game. Hence any height blend that plays DefectBot at some height, whatever its other branches, never ends CC with FairBot or any F_k; if unexploitable, it ends DD.

Status: derived ů author check only.

Verified.

- `checks/K28-c046_mixed.py`: Lean 4 core fk_cc_iff, fk_defects, db_height_blocks (SHA-pinned, no sorryAx, 3 mutations rejected); literal evaluator 6804 mixed F_k/DB/V_{n,r} blends (n1..4, r=1..n+1, k1,2, 243 labelings) vs self, F_1..F_3, CB: CC with F_m iff all-height cooperation; any DB label gives DD and F_m defects from s+m; DB-free blends match K22; 702 DB-labelled self-CC blends, none FB-CC — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K20

For finite S, $k, m \geq 1$: the F_k height blend of $J_{a,b,j}$ and $F_m = \Box^m p$ cooperate at every height iff $m \leq j$, else both eventually defect; so F_{j+1} separates every such blend, including every self-cooperating one, from FairBot and from each $\Box^{m'} p$.

Status: derived ů author check only.

Verified.

- `checks/K20-c046_j_xcheck.py`: Lean 4 core fm_db, fm_coop (m<=j: all-height CC, any S), fm_defect (j<m, finite S: eventual DD), fm_fm, separator; SHA-pinned, no sorryAx, 3 mutations rejected; exact engine: 53760 finite-S cells vs F_m (m<=7) CC iff m<=j; 1668 finite self-CC blends none FB-equivalent; K19 laws on 15360 blends — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K22

For $r, k, m \geq 1$ and any S, the F_k height blend of $V_{n,r}$ ($W_n = V_{n,1}$) and $F_m = \Box^m p$ cooperate at every height iff $m + r \leq n$ or all $h > \max(n, m)$ are selected, else both eventually defect; for finite S, $F_{\max(1,n-r+1)}$ separates the blend from every $\Box^{m'} p$.

Status: derived ů author check only.

Verified.

- `checks/K22-c046_v_fm.py`: Lean 4 core vfm_coop, vfm_defect, vfm_law (iff for arbitrary S), fm_fm, v_separator; SHA-pinned, no sorryAx, 3 mutations rejected; exact engine: 55296 cells (6912 V blends n2..5, r=1..n+1, k1..3, 128 selectors vs F_1..F_8) match the law; all blends unexploitable; 1010 finite self-CC blends DD with F_max(1,n-r+1), none FB-equivalent; hand examples W2{1}, V32{2}, W4{2} — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K24

An F_k height blend of $J_{a,b,j}$ ($k \geq 1$, cofinite S) is FairBot-equivalent iff $k = 1$ and all $h \geq 1$ are selected; else $R_h(q)$ (its DefectBot trace true at $h \geq 1$) or Y_{h_0} (unselected $h_0 \geq a + b + 1$) gets CC or DC from it and DD from FairBot.

Status: derived ů author check only.

Verified.

- `checks/K24-c046_jcof.py`: Lean 4 core db_trace, reader, cases3, fb_case, rq_blend, rq_fb, y_blend, y_fb (arbitrary a,b,j, k>=1, cofinite S); SHA-pinned, no sorryAx, 3 mutations rejected; exact engine on 7680 cofinite J blends: FB-equivalent iff k=1 and {1..5} in S (80), 7344 R_h(q)-separated (CC vs DD), 256 Y_h0-separated (DC vs DD); 6288 self-CC, 6208 separated incl. all 108 bank-unseparated; G11 hand example — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K26

For $n, r, k \geq 1$ and any finite or cofinite S, the F_k height blend of $V_{n,r}$ ($W_n = V_{n,1}$) is FairBot-equivalent iff $k = 1$ and all $h \geq 1$ are selected; else, for cofinite S, $R_h(q)$ gets CC from it and DD from FairBot, or Z_{n,h_0} gets DD from it and CC from FairBot.

Status: derived ů author check only.

Verified.

- `checks/K26-c046_vcof.py`: Lean 4 core db_trace, reader, cases3, fb_case, rq_blend (any unselected branch), rq_fb, z_blend, z_fb; SHA-pinned, no sorryAx, 3 mutations rejected; exact engine on 7680 V blends (n1..5, r=1..n+1, k1..3, 64 masks x finite/cofinite tail): FB-equivalent iff k=1 and S contains 1..5 cofinitely (40); 3664 R_h(q) CC/DD, 136 Z_{n,h0} DD/CC; 2522 cofinite self-CC, 2482 separated; W2 hand example unexploitable, self/FB/CB CC — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Discussion

The blends are a deliberately simple, deterministic way to mix agents: a formula without atoms decides, height by height, which agent to imitate. Probabilistic mixtures of FairBot-like agents have been studied elsewhere [23] and are not

covered here. The results are stated for the W, V and J families; whether similar window laws hold for every self-defecting unexploitable agent is open. The use of height guards to switch behaviour goes back to WaitFairBot in Barasz et al.

Provenance. Run `20261020-1600`, claims K14, K6, K16, K19, K9, K25, K23, K28, K20, K22, K24, K26, run-folder commit `e3d59e30bce37b58223f480d4f0d5a79bbff59ce`, published 2026-10-21.

10 How large must a program be to tell graded FairBots apart?

Setting

Two programs play a one-shot prisoner's dilemma, and each can read the other's source code. Following Barasz et al. [1], a *modal agent* decides by asking what can be proved about the opponent, written with the provability box $\Box$ of the logic GL: p means "the opponent cooperates with me" and q "the opponent cooperates with DefectBot". Graded FairBots are $F_k = \Box^k p$ (F_1 is FairBot). They are all unexploitable and cooperate with themselves and with each other, and a LessWrong discussion [26] lists them among the agents that share these properties with FairBot without being FairBot. We take guarded formulas built from p, q, $\bot$, $\neg$, $\wedge$, $\vee$, $\Box$ and measure size as the number of nodes. An opponent *separates* two agents if it gets a different outcome against them.

Result

- **Smallest separators.** Every opponent that separates F_k from F_{k+1} has size at least $k+5$, and the only one of size $k+5$ is $S_k = \Box(\neg q \vee \Box^k \bot)$ (up to the order of the disjuncts): it cooperates with F_k and mutually defects with F_{k+1}. Without q the minimum is $k+6$, attained by $T_k = \neg\Box(\neg p \vee \Box^k \bot)$.
- **Depth.** Every opponent of modal depth at most k gets the same outcome against F_k and F_{k+1}, so a separator needs depth at least $k+1$; S_k and T_k show this is tight.
- **Ingredients.** An opponent with no negation, or with no $\wedge/\vee$, gets the same outcome against every graded FairBot; without q the same holds for every boxed opponent and every opponent with at most one negation and at most one binary connective.

Why it matters

Graded FairBots look alike: each is unexploitable, cooperates with itself and with every other graded FairBot. These results say exactly how complex a

program must be to tell two neighbouring ones apart, and that the cost grows linearly with the grade.

Proved vs checked

All statements are proved for every k in Lean 4, in the height semantics of closed GL fixed points; the link to provability in arithmetic is Solovay's theorem [24], cited, not formalised (see also [11]). The uniqueness of T_k among q-free separators of size $k + 6$ is proved only for $k \leq 22$, by exhaustive search; the lower bound $k + 6$ holds for all k.

We believe this result is new. Our prior-art search (2026-10-21) covered:

- Barasz, LaVictoire, Christiano, Fallenstein, Herreshoff, Yudkowsky, "Robust Cooperation in the Prisoner's Dilemma: Program Equilibrium via Provability Logic" (arXiv 1401.5577): unexploitability (Section 2), rank and Theorem 4.10 (Section 4), WaitFairBot_K and the obstacle theorem (Section 5).
- LessWrong, "Is there only one FairBot?" (transhumanist_atom_understander, 29 Aug 2026) and comments: Zac Hatfield-Dodds shows the uniqueness conjecture holds up to modal depth 1 for rank-0 agents and fails from depth 2 (Box Box p; FairBot over PA+1 = Box(p or Box False); Box^k p for k >= 3); PrudentBot separates Box Box p from FairBot; the author notes Box Box p is not provably unexploitable in PA. No enumeration or counts.
- LessWrong / Alignment Forum wiki "Modal combat": DefectBot, CooperateBot, FairBot, PrudentBot, CliqueBot, TrollBot; no uniqueness result, no enumeration.
- LaVictoire et al. 2014 (MIPC); comment thread on "Robust Cooperation in the Prisoner's Dilemma" (LessWrong, on instability of FairBot equilibria); Hobson, "Yet More Modal Combat" (2021); Critch, parametric bounded Löb (arXiv 1602.04184); Critch, Dennis & Russell (arXiv 2208.07006); MIRI provability repository (github.com/machine-intelligence/provability, a modal-agent evaluator, no published census).
- Out of scope by direction: probabilistic blends (Oesterheld arXiv 2211.05057; Cooper et al. arXiv 2412.14570).
- Web searches for deterministic blends, behavioural distance and classifications of modal agents (2026-10-20/21); none means not found in this search.

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result extends Blackburn, de Rijke & Venema, Modal Logic (2001); "Is there only one FairBot?" (LessWrong, 2026). Our prior-art search (2026-10-21) covered:

- Barasz, LaVictoire, Christiano, Fallenstein, Herreshoff, Yudkowsky, "Robust Cooperation in the Prisoner's Dilemma: Program Equilibrium via Provability Logic" (arXiv 1401.5577): unexploitability (Section 2), rank and Theorem 4.10 (Section 4), WaitFairBot_K and the obstacle theorem (Section 5).
- LessWrong, "Is there only one FairBot?" (transhumanist_atom_understander, 29 Aug 2026) and comments: Zac Hatfield-Dodds shows the uniqueness conjecture holds up to modal depth 1 for rank-0 agents and fails from depth 2 (Box Box p; FairBot over PA+1 = Box(p or Box False); Box^k p for k >= 3); PrudentBot separates Box Box p from FairBot; the author notes Box Box p is not provably unexploitable in PA. No enumeration or counts.
- LessWrong / Alignment Forum wiki "Modal combat": DefectBot, CooperateBot, FairBot, PrudentBot, CliqueBot, TrollBot; no uniqueness result, no enumeration.
- LaVictoire et al. 2014 (MIPC); comment thread on "Robust Cooperation in the Prisoner's Dilemma" (LessWrong, on instability of FairBot equilibria); Hobson, "Yet More Modal Combat" (2021); Critch, parametric bounded Löb (arXiv 1602.04184); Critch, Dennis & Russell (arXiv 2208.07006); MIRI provability repository (github.com/machine-intelligence/provability, a modal-agent evaluator, no published census).
- Out of scope by direction: probabilistic blends (Oesterheld arXiv 2211.05057; Cooper et al. arXiv 2412.14570).
- Blackburn, de Rijke & Venema, Modal Logic (2001): n-bisimulation and modal depth.
- Web searches for deterministic blends, behavioural distance and classifications of modal agents (2026-10-20/21); none means not found in this search.

If you know of related work we missed, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was checked by scripts that run Lean proofs; the proof and check files are included so anyone can re-run them.

Claim K49

For all $k \geq 1$, every guarded separator of $F_k = \Box^k p$ and F_{k+1} has size $\geq k + 5$, and every size-$(k + 5)$ one is $S_k = \Box(\neg q \vee \Box^k \bot)$ up to disjunct order (CC with F_k, DD with F_{k+1}).

Status: derived ů author check only.

Verified.

- `checks/K49-c046_fk_sep.py`: Lean 4 core sizes, sk_ev, sep_coop (j<=k CC all heights), sep_defect (j>k), fk_fk, separator (all k; SHA-pinned, no sorryAx, 4 mutations rejected); exact engine over guarded classes of size <=11: k=1..6 no separator of size <=k+4, unique S_k at size k+5, k=7,8 none <=11; literal evaluator S_k vs F_j CC iff j<=k for k<=10, j<=12 — pass (Lean-proved, no sorryAx)

- `checks/K49-c046_fk_sep13.py`: guarded class counts 1..13 (36062, 100823 at 12, 13); exact engine k=1..8: no separator of F_k/F_{k+1} of size <=k+4, unique machine class S_k at size k+5; same Lean theorems as c046_fk_sep.py (sizes, sep_coop, sep_defect, fk_fk, separator; no sorryAx; 4 mutations rejected); literal evaluator S_k vs F_j CC iff j<=k — pass (Lean-proved, no sorryAx)

- `checks/K49-c046_fk_sep14.py`: guarded class counts 1..14 (100823 at 13, 279115 at 14; guarded-only products at top size); exact engine k=1..9: no F_k/F_{k+1} separator of size <=k+4, unique machine class S_k at size k+5; Lean sizes, sep_coop, sep_defect, fk_fk, separator (no sorryAx, 4 mutations rejected); literal evaluator S_k vs F_j CC iff j<=k — pass (Lean-proved, no sorryAx)

- `checks/K49-c046_fk_size40.py`: every reduced guarded formula of depth >=k+1: p,q sizes k+2..k+5 for k=1..40 (803710 solved), p-only sizes k+2..k+6 for k=1..22 (498344): no separator below k+5 / k+6, unique reduced separator S_k / T_k; full all-depth enumeration k<=2 reproduces minima without K58; bitmask games = naive literal evaluator on 1112 games — pass

- `checks/K49-c046_fk_depth.py`: K58 Lean same_outcome: no guarded opponent of modal depth <=k separates F_k/F_{k+1}, for every k — pass

- `checks/K49-c046_fk_shape.py`: Lean 4 core (SHA-pinned, includes K58 development): negfree_same (no negation: same outcome vs all F_j,F_j'), nobin_same (no &/|: same outcome vs all F_j,F_j', j,j'>=1), size_bound (k>=1: separator has ~, binary node, md>=k+1, size>=k+5), sk_size; no sorryAx; 4 mutations rejected; literal evaluator: 4333 such formulas of size<=8 same vs F_1..F_5, all 414 separator cells (size<=9, k<=4) have ~ and a binary node — pass (Lean-proved, no sorryAx)

- `checks/K49-c046_fk_unique.py`: Lean 4 core unique_sep (all k>=1: guarded size-(k+5) separator of F_k/F_{k+1} is S_k up to disjunct order), code_same, shapeA/B/C, size_bound, sk_codes; SHA-pinned; no sorryAx/ofReduceBool; 4 mutations rejected; literal evaluator S_k k<=10 and exhaustive unreduced guarded syntax of size 6,7: only S_1,S_2 (both orders) separate — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K56

For all $k \geq 1$, every q-free guarded separator of $F_k = \Box^k p$ and F_{k+1} has size $\geq k+6$, attained by $T_k = \neg\Box(\neg p \vee \Box^k \bot)$ (DD with F_k, exploited by F_{k+1}); for $k \leq 22$ every size-$(k+6)$ one is T_k up to disjunct order.

Status: derived ů author check only.

Verified.

- `checks/K56-c046_fk_psep17.py`: p-only guarded class counts 1..17 (K51 list to 15, then 35721, 74728); exact engine k=1..11: no p-only F_k/F_{k+1} separator of size <=k+5, unique machine class T_k at size k+6 with F_k DD and F_{k+1} (D,C); k=12 none <=17; Lean tsize, tk_ev, t_low, t_high, psep for all k (no sorryAx, 4 mutations rejected); literal evaluator k<=12, j<=14 — pass (Lean-proved, no sorryAx)

- `checks/K49-c046_fk_size40.py`: every reduced guarded formula of depth >=k+1: p,q sizes k+2..k+5 for k=1..40 (803710 solved), p-only sizes k+2..k+6 for k=1..22 (498344): no separator below k+5 / k+6, unique reduced separator S_k / T_k; full all-depth enumeration k<=2 reproduces minima without K58; bitmask games = naive literal evaluator on 1112 games — pass

- `checks/K49-c046_fk_depth.py`: K58 Lean same_outcome: no guarded opponent of modal depth <=k separates F_k/F_{k+1}, for every k — pass

- `checks/K56-c046_fk_psize.py`: Lean 4 core (SHA-pinned, includes K58 and K49 shape developments): stab_box, stab_top, pone_same, one_one_same (q-free, <=1 neg and <=1 binary: same outcome vs all F_j,F_j', j,j'>=1), psize_bound (q-free separator of F_k,F_{k+1}: size >= k+6, all k>=1), tk_size; no sorryAx; 4 mutations rejected; literal evaluator 8890 small and 40758 boxed q-free formulas same vs F_1..F_6, T_k DD/CD k<=6 — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K58

For every $k \geq 1$, every guarded opponent O in p, q of modal depth $\leq k$ gets the same eventual outcome against $F_k = \Box^k p$ and F_{k+1}; so every separator has depth $\geq k+1$ (size $\geq k+2$), tight for S_k (K49) and T_k (K56).

Status: derived ů author check only.

Verified.

- `checks/K49-c046_fk_depth.py`: Lean 4 core (SHA-pinned) same_outcome for all k and guarded O with md<=k, separator_bound (md>=k+1, size>=k+2), sk_depth=k+1; no sorryAx; 5 mutations rejected; exact engine k=1..6: no depth<=k class of size<=12 separates, depth k+1 control finds S_k; literal evaluator 3200 random depth<=k opponents, exact K40 horizon — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K59

For guarded O in p, q with no negation, or with no $\wedge/\vee$, and all $j, j' \geq 1$: O gets the same eventual outcome against $F_j = \Box^j p$ and $F_{j'}$ (Lean, height semantics).

Status: derived ů author check only.

Verified.

- `checks/K49-c046_fk_shape.py`: Lean 4 core (SHA-pinned, includes K58 development): negfree_same (no negation: same outcome vs all F_j,F_j'), nobin_same (no &/|: same outcome vs all F_j,F_j', j,j'>=1), size_bound (k>=1: separator has ~, binary node, md>=k+1, size>=k+5), sk_size; no sorryAx; 4 mutations rejected; literal evaluator: 4333 such formulas of size<=8 same vs F_1..F_5, all 414 separator cells (size<=9, k<=4) have ~ and a binary node — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K60

In L without q (guarded), every boxed opponent $\Box W$, and every opponent with at most one $\neg$ and at most one $\wedge/\vee$, gets the same eventual outcome against $F_j = \Box^j p$ and $F_{j'}$ for all $j, j' \geq 1$ (Lean, height semantics).

Status: derived ů author check only.

Verified.

- `checks/K56-c046_fk_psize.py`: Lean 4 core (SHA-pinned): stab_box, stab_top, pone_same (q-free O boxed or with top-level boxes negation-free or binary-free: same outcome vs all F_j,F_j', j,j'>=1), one_one_same (<=1 neg, <=1 binary), psize_bound, tk_size; no sorryAx; 4 mutations rejected; literal evaluator 8890 small and 40758 boxed q-free formulas same vs F_1..F_6 — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Discussion

The depth bound uses the standard back-and-forth (n-bisimulation) method of modal logic [10]; the new part is its application to games between modal agents. The results concern separating graded FairBots from each other; how large a separator must be for other pairs of agents (for example the gate-family members of earlier findings) is open, as is a numerical bound on separator size at fixed modal depth.

Provenance. Run `20261020-1600`, claims K49, K56, K58, K59, K60, run-folder commit `e3d59e30bce37b58223f480d4f0d5a79bbff59ce`, published 2026-10-21.

11 Unexploitable modal agents that cooperate with themselves: a census

Setting

Two programs play a one-shot prisoner's dilemma, and each can read the other's source code. Following Barasz et al. [1], a *modal agent* decides by asking what can be proved about the opponent, written with the provability box $\Box$ of the logic GL: p means "the opponent cooperates with me" and q "the opponent cooperates with DefectBot". FairBot is $\Box p$: it cooperates exactly when it can prove that the opponent cooperates with it. An agent is *unexploitable* if no modal opponent ever ends up defecting while the agent cooperates, and *self-cooperating* if two copies cooperate. We take guarded formulas (every atom under a box) built from p, q, $\bot$, $\neg$, $\wedge$, $\vee$, $\Box$, measure size as the number of nodes, and call two agents behaviourally equal when they make the same moves against every opponent. Agents using only p are the rank-0 agents of Barasz et al.

A LessWrong post [26] asked whether every rank-0 agent that is unexploitable and self-cooperating is FairBot. A comment there shows that the answer is yes up to modal depth 1 and no from depth 2, with examples such as $\Box\Box p$, FairBot over PA+1 and $\Box^k p$.

Result

- **Rank 0.** Among rank-0 agents of size at most 15 there are exactly 5031 classes that are unexploitable and self-cooperating (4374 of them also cooperate with FairBot and CooperateBot). Only $\Box p$ itself is FairBot; every other class is told apart from FairBot by an explicit opponent.
- **With q.** Allowing q, up to size 14 there are 26449 such classes (16756 up to behaviour); 387 of them behave exactly like FairBot against every opponent, and each of the others is told apart from FairBot by an explicit opponent.

- **Why some different formulas are FairBot.** Two agents behave identically against every opponent exactly when they have the same moves against DefectBot and agree on every input the opponent can actually produce: until the agent first cooperates, the opponent's move against it equals its move against DefectBot. So, for example, $\Box(p \vee (q \wedge \Box\bot))$ is behaviourally FairBot.

- **A small example that is not FairBot.** $\neg\Box\bot \wedge \Box(p \vee \Box\bot)$ (size 9) is unexploitable and cooperates with itself and with CooperateBot, yet mutually defects with FairBot; it is the agent $\mathrm{WaitFairBot}_1$ of Barasz et al. Every smaller rank-0 agent that is unexploitable and self-cooperating cooperates with FairBot.

Why it matters

The LessWrong question was answered with a few examples. These counts make the answer quantitative: among small agents, FairBot is one of thousands of unexploitable self-cooperators, and for each one there is a concrete opponent that tells it apart from FairBot. The criterion for behavioural equality explains which syntactically different agents are nevertheless FairBot.

Proved vs checked

The equality criterion and the FairBot-equivalence of $\Box(p\vee(q\wedge\Box\bot))$ are proved in Lean 4, in the height semantics of closed GL fixed points; the link to provability in arithmetic is Solovay's theorem [24], cited, not formalised (see also [11]). The counts are exhaustive computations up to the stated sizes, with every separating opponent recorded and re-solved by the checks. The minimality statement for the size-9 agent is an exhaustive search over smaller rank-0 agents.

This result extends Barasz et al. (arXiv 1401.5577): modal agents, unexploitability, rank, and WaitFairBot_K; "Is there only one FairBot?" (LessWrong, 2026), with Zac Hatfield-Dodds's comment answering it to modal depth 1 and giving examples. Our prior-art search (2026-10-21) covered:

- Barasz, LaVictoire, Christiano, Fallenstein, Herreshoff, Yudkowsky, "Robust Cooperation in the Prisoner's Dilemma: Program Equilibrium via Provability Logic" (arXiv 1401.5577): unexploitability (Section 2), rank and Theorem 4.10 (Section 4), WaitFairBot_K and the obstacle theorem (Section 5).

- LessWrong, "Is there only one FairBot?" (transhumanist_atom_understander, 29 Aug 2026) and comments: Zac Hatfield-Dodds shows the uniqueness conjecture holds up to modal depth 1 for rank-0 agents and fails from depth 2 (Box Box p; FairBot over PA+1 = Box(p or Box False); Box^k p for k >= 3); PrudentBot separates Box Box p from FairBot; the author notes Box Box p is not provably unexploitable in PA. No enumeration or counts.

- LessWrong / Alignment Forum wiki "Modal combat": DefectBot, CooperateBot, FairBot, PrudentBot, CliqueBot, TrollBot; no uniqueness result, no enumeration.
- LaVictoire et al. 2014 (MIPC); comment thread on "Robust Cooperation in the Prisoner's Dilemma" (LessWrong, on instability of FairBot equilibria); Hobson, "Yet More Modal Combat" (2021); Critch, parametric bounded Löb (arXiv 1602.04184); Critch, Dennis & Russell (arXiv 2208.07006); MIRI provability repository (github.com/machine-intelligence/provability, a modal-agent evaluator, no published census).
- Out of scope by direction: probabilistic blends (Oesterheld arXiv 2211.05057; Cooper et al. arXiv 2412.14570).
- Web searches for deterministic blends, behavioural distance and classifications of modal agents (2026-10-20/21); none means not found in this search.

If you know of related work we missed, please open an issue.
We believe this result is new. Our prior-art search (2026-10-21) covered:

- Barasz, LaVictoire, Christiano, Fallenstein, Herreshoff, Yudkowsky, "Robust Cooperation in the Prisoner's Dilemma: Program Equilibrium via Provability Logic" (arXiv 1401.5577): unexploitability (Section 2), rank and Theorem 4.10 (Section 4), WaitFairBot_K and the obstacle theorem (Section 5).
- LessWrong, "Is there only one FairBot?" (transhumanist_atom_understander, 29 Aug 2026) and comments: Zac Hatfield-Dodds shows the uniqueness conjecture holds up to modal depth 1 for rank-0 agents and fails from depth 2 (Box Box p; FairBot over PA+1 = Box(p or Box False); Box^k p for k >= 3); PrudentBot separates Box Box p from FairBot; the author notes Box Box p is not provably unexploitable in PA. No enumeration or counts.
- LessWrong / Alignment Forum wiki "Modal combat": DefectBot, CooperateBot, FairBot, PrudentBot, CliqueBot, TrollBot; no uniqueness result, no enumeration.
- LaVictoire et al. 2014 (MIPC); comment thread on "Robust Cooperation in the Prisoner's Dilemma" (LessWrong, on instability of FairBot equilibria); Hobson, "Yet More Modal Combat" (2021); Critch, parametric bounded Löb (arXiv 1602.04184); Critch, Dennis & Russell (arXiv 2208.07006); MIRI provability repository (github.com/machine-intelligence/provability, a modal-agent evaluator, no published census).
- Out of scope by direction: probabilistic blends (Oesterheld arXiv 2211.05057; Cooper et al. arXiv 2412.14570).

- Web searches for deterministic blends, behavioural distance and classifications of modal agents (2026-10-20/21); none means not found in this search.

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was partly checked by scripts that run Lean proofs (claims K11, K18) and partly checked by script (claims K51, K53, K2); the proof and check files are included so anyone can re-run them.

Claim K51

In rank-0 L (atom p, guarded, size ≤ 15) exactly 5031 classes are unexploitable and self-cooperating, 4374 of them cooperating with FairBot and CooperateBot; only $\Box p$ is FairBot, every other one is separated from it by an explicit opponent.

Status: derived ů author check only.

Verified.

- `checks/K51-c046_ponly15.py`: guarded p-only class counts per size 1..15; 15092 exploitable self-cooperators each exploited by a re-solved letterless opponent; 5031 unexploitable self-cooperators (per-size list); 5030 non-FB each separated from []p by a constructed re-solved opponent; FB-defecting 4,8,16,33,80,171,345 (sizes 9..15), CB-defecting 2,4,8,16,41,87,181, FB&CB-cooperating 4374; depth-1 reproduction; K1 prefix counts — pass
- `checks/K51-c046_top.py`: K51 engine enumerated to size 13 with and without primitive top (size 1): guarded counts differ only at sizes 1,2; unexploitable self-cooperator counts per size identical (1202); top itself exploitable — pass

Prior art. Partial prior art.

Claim K53

In L with atoms p, q (guarded, size ≤ 14), 26449 machine classes (16756 behaviour classes) are unexploitable and self-cooperating; 387 of them behave as FairBot, and each other one gets a different outcome than FairBot against an explicit opponent.

Status: derived ů author check only.

Verified.

- `checks/K53-c046_qfull14.py`: guarded class counts 1..14 (279115 at 14); 249390 self-cooperators, 222941 exploited by re-solved explicit opponents; 26449 unexploitable (per-size list), 16756 behaviour classes; FairBot class

387 machine classes with per-size list; 26062 others separated from []p by re-solved opponents; rank-0 2468 (=K51 prefix); 1058 not CC with FB (smallest size 9), 15306 CC with FB and CB; literal evaluator cross-check — pass

Prior art. Partial prior art.

Claim K11

$X = \Box(p \vee (q \wedge \Box\bot))$ (size 7) has the same moves as FairBot $\Box p$ at every height against every modal opponent, since an opponent's height-0 move against any agent equals its move against DefectBot.

Status: derived ůa author check only.

Verified.

- `checks/K11-c046_xfb_lean.py`: Lean 4 core theorem X_same_as_FairBot for every causal opponent operator (fixed points coincide at all heights), no sorryAx; 3 mutations rejected — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K18

For causal agents A, A' in height semantics: they make the same moves in every game against every causal opponent iff their DefectBot traces are equal and they agree on all realizable words ($p = q$ up to the first cooperation of A or its DefectBot trace).

Status: derived ůa author check only.

Verified.

- `checks/K18-c046_realizable_lean.py`: Lean 4 core theorems game_exists, inputs_realizable, same_behaviour, separation, behaviour_iff, X_beheq_FB; SHA-pinned source; no sorryAx; 4 mutations rejected — pass (Lean-proved, no sorryAx)
- `checks/K18-c046_qfull12.py`: behav() realizable-word machines: 2574 classes, FairBot class has 86 machine classes with identical realizable-word machine to []p (consequence: FairBot behaviour against every opponent via same_behaviour) — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K2

$A = \neg\Box\bot \wedge \Box(p \vee \Box\bot)$ (size 9) is unexploitable by every modal opponent, cooperates with its copy and with CooperateBot, yet mutually defects with FairBot; every rank-0 unexploitable self-cooperator of size ≤ 8 cooperates with FairBot.

Status: derived ů author check only.

Verified.

- `checks/K2-c046_ponly12.py`: A and ~[]p&[](p|[]F) are among the 591 unexploitable self-cooperators, A vs A = CC, A vs []p = DD, A vs ~F = CC; all such agents not cooperating with FB have size >= 9 — pass

Prior art. Partial prior art.

Discussion

The counts depend on the agent language and the size measure; they say how common non-FairBot unexploitable self-cooperators are among small formulas, not in any absolute sense. "Unexploitable" is about what actually happens in play (truth in the standard model, as in Barasz et al.), not about what Peano arithmetic can prove about the agent; the LessWrong discussion points out that the two differ. Open questions: how the counts grow with size, and whether the non-FairBot classes fall into a small number of families.

Provenance. Run `20261020-1600`, claims K51, K53, K11, K18, K2, run-folder commit `e3d59e30bce37b58223f480d4f0d5a79bbff59ce`, published 2026-10-21.

12 Round robins of modal agents in the Stag Hunt

Setting

In the Stag Hunt both players do best by hunting stag together; hunting stag alone pays nothing, and hare is a safe middle option. We write the payoffs as stag/stag a, stag against hare s, hare against stag t, hare/hare d, with $a > t > d > s$. The players are modal agents in the sense of Barasz et al. [1]: programs that decide by what can be proved about the opponent, with p = "the opponent hunts stag against me" and q = "the opponent hunts stag against HareBot". FairBot $\Box p$ hunts stag exactly when it can prove the opponent does. We take all 7,561 behaviourally distinct guarded agents of size at most 10 and let every agent play every agent, itself included, with equal weight (a round robin).

Result

- **FairBot never wins.** Normalising $a = 1$, $s = 0$, the triangle of strict Stag Hunts $0 < d < t < 1$ splits into exactly six regions, each won by a single agent, and FairBot wins none of them. The agent $A_3 = \Box(p \vee \Box^3\neg(p \vee q))$ wins on 44.6% of the area, including every game with payoffs $(4, 0, t, 2)$ for $2 < t < 4$ and the game $(3, 0, 2, 1)$. At $(4, 0, 3, 2)$ it scores 23,411 against FairBot's 21,719.

- **Why it beats FairBot.** A_3 reaches stag/stag with 4,444 of the 4,459 agents that hunt stag against StagBot and with none of the others; it gains on the first group and loses on the second, for a net gain of 1,692 points over FairBot at $(4, 0, 3, 2)$.

- **Families behind the winners.** For every k, $A_k = \Box(p \vee \Box^k\neg(p \vee q))$ hunts stag against HareBot (and is left alone there), coordinates with StagBot, with every A_m and with every $F_j = \Box(\Box^j\bot \vee p)$ (FairBot over a stronger system); the lenient FairBots $R_k = \Box(p \vee \Box^k p)$ are safe and coordinate with $\neg\Box^j\bot$ exactly when $j \leq k$.

- **The winner depends on the size bound.** In the same census the score of R_k rises strictly with k up to $k = 8$ and that of A_k up to $k = 5$, so longer members of both families score higher than the census winners. The specific winners above are therefore artefacts of the size bound; what holds across the families is that lenient agents beat FairBot in this round robin.

Why it matters

Round robins are a natural way to compare programs that read each other's code, and FairBot is often taken as the reference cooperative agent. In the Stag Hunt, where cooperating with a defector costs less than in the prisoner's dilemma, agents that are more willing to hunt stag do better than FairBot against a broad population, and exact payoff breakpoints show where each agent takes over.

Proved vs checked

The laws for A_k and R_k are proved for all indices in Lean 4, in the height semantics of closed GL fixed points; the link to provability in arithmetic is Solovay's theorem [24], cited, not formalised (see also [11]). Scores, regions and breakpoints are exact rational computations over the stated census.

We believe this result is new. Our prior-art search (2026-10-21) covered:

- Barasz, LaVictoire et al., arXiv 1401.5577.

- LessWrong / Alignment Forum wiki "Modal combat" and "Is there only one FairBot?" (2026).

- Mennen, "Modal combat for games other than the prisoner's dilemma" (2017).
- Todorov, ten Napel & Müller, "Parametric Open Source Games", arXiv 2606.27068.
- Tennenholtz (2004), program equilibrium (background).
- Web searches for modal agents in the Stag Hunt and for tournaments of modal agents (2026-10-21); none means not found in this search.

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was partly checked by scripts that run Lean proofs (claims K8, K3) and partly checked by script (claims K15, K6, K14, K5, K7, K4, K17, K18); the proof and check files are included so anyone can re-run them.

Claim K15

Normalising $a = 1, s = 0$, the strict Stag Hunt triangle $0 < d < t < 1$ splits into exactly 6 census (size ≤ 10) round-robin winner regions, each one agent; A_3 wins on 44.6% of the area and $R_6 = \Box(p \vee \Box^6 p)$ near $t, d \to 1$; FairBot nowhere.

Status: derived ů author check only.

Verified.

- `checks/K15-c046_sh_triangle.py`: 7561 classes re-enumerated (own engine); K13/K14 values; 3309 score planes; exact Fraction clipping gives 6 winner polygons with total area exactly 1/2, one agent each, exact areas and A3 vertices; unique exact argmax at 7 interior points; 6 GLS self-certificates — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K6

In the 7,561-class guarded-language census of size ≤ 10, $A_3 = \Box(p \vee \Box^3 \neg(p \vee q))$ uniquely maximizes round-robin score $21950 + 487t$ for every $2 < t < 4$, with payoffs $(SS, SH, HS, HH) = (4, 0, t, 2)$.

Status: derived ů author check only.

Verified.

- `checks/K6-stag_payoffs.py`: Pinned solver source; exact census counts; all ordered cells solved; GLS certificates for A3/self, A3/HareBot, A3/FairBot and FairBot/self; exact outcome counts, transitions, and upper envelopes for both payoff slices. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K14

At $(SS, SH, HS, HH) = (3, 0, 2, 1)$, $A_3 = \Box(p \vee \Box^3 \neg(p \vee q))$ uniquely maximizes the equally weighted round robin over the 7561 guarded size-≤ 10 census classes, scoring 16393 versus FairBot's 14158.

Status: derived ů author check only.

Verified.

- `checks/K14-todorov_safety.py`: SHA256-pinned solver; all 7561-class cells re-solved; unique winner A3 at (3,0,2,1), score 16393 versus FairBot 14158; 7561 winner cells independently re-solved; 4 GLS certificates; 1693 census-safe classes, 517 self-stag. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K5

For $(SS, SH, HS, HH) = (4, 0, 3, h)$ and $0 < h < 4$, the size-≤ 10 modal census has six winner intervals, separated by 116/1423, 15/22, 2537/1001, 4079/1071, and 463/116.

Status: derived ů author check only.

Verified.

- `checks/K6-stag_payoffs.py`: Pinned solver source; exact census counts; all ordered cells solved; GLS certificates for A3/self, A3/HareBot, A3/FairBot and FairBot/self; exact outcome counts, transitions, and upper envelopes for both payoff slices. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K7

In L up to size 10 with Stag Hunt $(4, 0, 3, 2)$, $A_3 = \Box(p \vee \Box^3 \neg(p \vee q))$ ends stag/stag with exactly 4444 of the 4459 opponents that hunt stag vs StagBot and with none of the other 3102; vs FairBot it gains +3848 on the first group and loses 2156 on the second.

Status: derived ů author check only.

Verified.

- `checks/K7-c046_xreview_k4k6.py`: 7561 classes; A3/FB counts and K4 transitions; K5/K6 envelopes; StagBot split 4459/3102, A3 SS = 4444 all in first group, 15 HS exceptions of size >=9 with []^4, gains +3848/-2156; 60488 cells re-solved independently; 4 GLS certificates — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K4

At Stag Hunt payoffs $(4, 0, 3, 2)$ in the size-≤ 10 census, A_3 gains 1,692 points over FairBot: 1,643 opponents change $HH \to SS$ and 562 change $HS \to SS$, offset by other outcome changes.

Status: derived ů author check only.

Verified.

- `checks/K6-stag_payoffs.py`: Pinned solver source; exact census counts; all ordered cells solved; GLS certificates for A3/self, A3/HareBot, A3/FairBot and FairBot/self; exact outcome counts, transitions, and upper envelopes for both payoff slices. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K8

For all $k, j, m \geq 0$, A_k plays stag/hare vs HareBot and stag/stag vs StagBot, A_m, and $F_j = \Box(\Box^j \bot \vee p)$. Against $PB_j = \Box p \wedge \Box(\Box^{j+1} \bot \vee \neg q)$, it plays stag/hare for $k = 0$ and hare/hare for $k \geq 1$.

Status: derived ů author check only.

Verified.

- `checks/K8-ak_laws.py`: Seven core Lean theorems for all indices and arbitrary solution traces; no sorryAx; incorrect strict bound rejected; 261 independent height cells; eight GLS fixed-point certificates; pinned prior solver source. — pass (Lean-proved, no sorryAx)
- `checks/K8-ak_laws.py`: 11 core Lean theorem reports, no sorryAx; faithful PB guard equivalent to original P_(j+1) when q=true; all-index A/PB outcomes; false mutation rejected; 342 height cells; 12 GLS certificates. — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K3

For all $k \geq 0$, $R_k = \Box(p \vee \Box^k p)$ (R_0 = FairBot) never ends hunting stag against an opponent that ends hunting hare, and in the Stag Hunt ends stag/stag with $E_j = \neg\Box^j \bot$ iff $j \leq k$ (else hare vs stag) and with $N_n = \Box(\Box^n \bot \vee \neg q)$ iff $n > k$ (else hare/hare).

Status: derived ů author check only.

Verified.

- `checks/K3-c046_lenient_laws.py`: Lean (17 theorems, no sorryAx): rk_safe, rr_pair, rk_db, rk_E (stag/stag iff j<=k), rk_N (stag/stag iff n>k, else hare/hare), modal bridges; false variant rejected; height tables k<=7, j,n<=8; 14 GLS certificates — pass (Lean-proved, no sorryAx)
- `checks/K3-c046_relabel_gk.py`: Lean re-proves rk_E, rk_N, rk_Z, rk_db for all indices alongside gk_E, gk_N (same E_j/N_n laws for G_k); no sorryAx; false bound rejected; 588 height cells; 10 GLS certificates — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K17

Against the 7561 size-
le10 census agents, $R_k =$
$Box($p
lor
$Box^k p$) has round-robin score strictly increasing in k for 0
lek
le8 and constant for 8
lek
le12, for all payoffs with $a > t, a > d$; so R_6's K15 region reflects the size bound (size $k + 4$).

Status: derived ů author check only.

Verified.

- `checks/K17-c046_rk_artifact.py`: 7561 classes re-enumerated; R_k counts k=0..12 (machine solver = height solver for k<=6), SH=0; SS up, HS/HH non-increasing, strict until k=8 then constant; 200 exact random payoffs: score strictly increasing to k=8, R_8-R_6=4(a-t); R_6->R_7 differs on ~[]^7F,~[]^7~p,~[]^7~q, R_7->R_8 on ~[]^8F; 4 GLS certificates — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K18

Against the 7561 size-≤ 10 census agents, $A_k = \Box(p \vee \Box^k \neg(p \vee q))$ (size $k + 7$) has 543 SH and 2087 HH for $1 \leq k \leq 10$; its score rises strictly for $k \leq 5$ (by $\Delta SS(a - t)$) and is constant after, so $A_5 - A_3 = 15(a - t)$ whenever $a > t$.

Status: derived ů author check only.

Verified.

- `checks/K18-c046_ak_artifact.py`: 7561 classes re-enumerated; A_k counts k=1..10 (machine = height solver for k<=3, A_4.. not census members); SH 543, HH 2087 fixed; only HS->SS for k>=2; 200 exact random payoffs: strict increase to k=5, constant to k=10, A_5-A_3=15(a-t); A_5 SS set = 4459 stag-vs-StagBot agents; 12 then 3 listed flips; 5 GLS certificates — pass

Prior art. Possibly new: no prior art found in the owner's search.

Discussion

Round-robin results depend on the opponent population: here all small formulas with equal weight, which over-represents some behaviours. The scores are aggregate quantities, not equilibrium thresholds; they are not comparable to the learning-dynamics threshold of Todorov et al. for parametric (non-modal) agents [25]. Open: whether the scores of A_k and R_k keep their order in larger censuses, and which agent wins in the limit.

Provenance. Run `20261021-0517`, claims K15, K6, K14, K5, K7, K4, K8, K3, K17, K18, run-folder commit `580565ffd3b3f1a8ca3ffa0407c949c8d724865c`, published 2026-10-21.

13 Safe modal agents that coordinate in the Stag Hunt: size 13 is the minimum

Setting

In the Stag Hunt two players each choose stag or hare: both hunting stag pays most, hunting stag alone pays least, and hare is a safe middle option. Here the players are programs that read each other's source code. Following Barasz et al. [1], a *modal agent* decides by asking what can be proved about the opponent, written with the provability box $\Box$ of the logic GL: p means "the opponent hunts stag against me" and q "the opponent hunts stag against HareBot", the agent that always hunts hare. Call an agent *safe* if no modal opponent ever ends up hunting hare while the agent hunts stag (the Stag Hunt reading of unexploitability). Two simple probes test willingness to coordinate: $E_1 = \neg\Box\bot$, which hunts stag unconditionally in practice, and $N_1 = \Box(\Box\bot \vee \neg q)$, which hunts stag against agents that do not hunt stag against HareBot. We take guarded formulas built from p, q, $\bot$, $\neg$, $\wedge$, $\vee$, $\Box$ and measure size as the number of nodes.

Result

- **No small safe coordinator.** Among the 56,503 agent classes of size at most 12, exactly 356 reach stag/stag with themselves, with E_1 and with N_1, and every one of them is exploited by some opponent of size at most 10. So no safe agent of size at most 12 has all three coordination properties.
- **Size 13 is enough.** The agent $X_1 = \Box(p \vee \Box p) \wedge (\Box\bot \vee \neg\Box\neg p)$ of size 13 is safe against every modal opponent and reaches stag/stag with itself, with FairBot, with E_1 and with N_1. More generally, the family $X_k = \Box(p \vee \Box^k p) \wedge (\Box\bot \vee \neg\Box\neg p)$ is safe for every $k \geq 1$, coordinates with itself and with FairBot, with E_j exactly when $j \leq k$, and with every N_n, $n \geq 1$.
- **Census safety counts.** Among the 7,561 classes of size at most 10, exactly 1,693 avoid being left hunting stag alone against every agent in that census, and 517 of these hunt stag with themselves; tested against all opponents of size at most 12, only one of the 517, $\Box(p \vee (q \wedge \neg\Box^3 q))$, is exploited (by an opponent of size 11).

Why it matters

Safe agents in the Stag Hunt must refuse to hunt stag unless they can be sure of a partner, which pulls against coordinating with simple partners such as E_1 and N_1. The result locates the smallest size at which safety and these coordination properties can be combined, and gives an explicit infinite family that does it.

Proved vs checked

The laws for X_k, including safety against every modal opponent, are proved for all k in Lean 4, in the height semantics of closed GL fixed points; the link to provability in arithmetic is Solovay's theorem [24], cited, not formalised (see also [11]). The non-existence below size 13 and the census counts are exhaustive computations up to the stated sizes, with every exploiting opponent recorded and re-solved.

We believe this result is new. Our prior-art search (2026-10-21) covered:

- Barasz, LaVictoire et al., arXiv 1401.5577.
- LessWrong / Alignment Forum wiki "Modal combat" and "Is there only one FairBot?" (2026).
- Mennen, "Modal combat for games other than the prisoner's dilemma" (2017).
- Todorov, ten Napel & Müller, "Parametric Open Source Games", arXiv 2606.27068.

- Tennenholtz (2004), program equilibrium (background).
- Web searches for modal agents in the Stag Hunt and for tournaments of modal agents (2026-10-21); none means not found in this search.

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was partly checked by scripts that run Lean proofs (claims K9) and partly checked by script (claims K16, K13, K19); the proof and check files are included so anyone can re-run them.

Claim K16

In guarded L of syntax-node size ≤ 12, exactly 356 of 56,503 classes get stag/stag with self, E_1, and N_1; every one is exploited by a size-≤ 10 opponent. Hence no universally safe agent of size ≤ 12 has all three coordination properties.

Status: derived ů author check only.

Verified.

- `checks/K16-no_safe12.py`: SHA256-pinned full enumerator; 56503 classes through size 12; independent machine and formula filters give exactly 356 self/E1/N1 candidates; each has re-solved size<=10 exploitation witness; 362 GLS certificates. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K9

For all $k \geq 1$, $X_k = \Box(p \vee \Box^k p) \wedge (\Box\bot \vee \neg\Box\neg p)$ never ends hunting stag against an eventual hare hunter, ends stag/stag with itself, FairBot and every R_m, D_m, X_m, with E_j ($j \geq 1$) iff $j \leq k$ and with N_n iff $n \geq 1$.

Status: derived ů author check only.

Verified.

- `checks/K9-c046_patient_laws.py`: Lean 16 theorems (xk_safe, xk_self, xx/xr/xd_pair, xk_db, xk_E, xk_N, xk_Z, xk_modal,...), no sorryAx, false bound rejected; 452 height cells k<=7; 12 GLS certificates; census size<=10: 1693 safe, 517 safe self-stag, 230 SS with E_1, 75 with N_1, 0 with both — pass (Lean-proved, no sorryAx)

Prior art. Possibly new: no prior art found in the owner's search.

Claim K13

Among the 7561 guarded L classes of syntax-node size ≤ 10, exactly 1693 avoid stag/hare against every census opponent; exactly 517 of these hunt stag with themselves. These are finite-census safety counts.

Status: derived ů author check only.

Verified.

- `checks/K13-todorov_safety.py`: SHA256-pinned solver; all 7561-class cells re-solved; unique winner A3 at (3,0,2,1), score 16393 versus FairBot 14158; 7561 winner cells independently re-solved; 4 GLS certificates; 1693 census-safe classes, 517 self-stag. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Claim K19

Of the 517 size-≤ 10 census-safe self-stag agents, exactly one is exploited by a size-≤ 12 opponent: $V = \Box(p \vee (q \wedge \neg\Box^3 q))$, by $B = \neg\Box^2 p \vee \Box(p \wedge \Box^2 q)$ of minimum size 11. The other 516 survive this finite test.

Status: derived ů author check only.

Verified.

- `checks/K19-safety12.py`: Pinned full enumeration: 56503 classes; 7561-class matrix recovers 1693/517; 25303014 extension cells at exact joint fixed states; exactly V refuted, 516 survive; minimum exploiter size11; 7561 independent V row cells; exploitation and self-play GLS certificates. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Discussion

The census counts are finite statements: "safe" there means safe against the agents in the census, and tests against larger opponents can still find exploits, as they did for one agent. The size-13 agent is safe against every modal opponent by proof. Safety is the same outcome condition as unexploitability in the prisoner's dilemma; what changes in the Stag Hunt is which coordination properties one wants alongside it.

Provenance. Run `20261021-0517`, claims K16, K9, K13, K19, run-folder commit `580565ffd3b3f1a8ca3ffa0407c949c8d724865c`, published 2026-10-21.

14 Nominal-free proofs of the theorems Benzmüller's detector flags

Setting

Benzmüller and Scott's *Notes* prove variants of Gödel's ontological argument in a shallow embedding of higher-order modal logic in classical higher-order logic [8]. Benzmüller's *Proofs Without Nominals* [5] (v4) points out that the embedding's quantifiers also range over *nominals*, properties like $\lambda u.\, u = w$ that pick out a single world. A proof that substitutes one proves a theorem of the embedding but not necessarily of the modal object language $\mathcal{L}$. v4 ships a detector that flags such "hybrid" proofs in two modes (witness and whole-term), and lists the theorems of its Lean modules it could only prove with a flagged step, among them two of its stated open questions: whether the equivalence of the restricted conjunction axiom Ax1GenTwo with the unrestricted Ax1Gen has a nominal-free proof in variants 2 and $3'$, and whether Gödel's lemma L follows under the rigid reading Ax1GenR with no condition on positivity.

Result

For each flagged theorem below we give a new proof of the *identical* statement, from the same module's definitions, that v4's own detector passes in both modes (full names):

- **The equivalence question:** `Ax1Gen_of_Two` and `Two_equivalent`, with `P_of_conj`, `loc_of_conj` and the related lemmas, in `Ax1GenTwoVariant2` and `Ax1GenTwoVariant3Plain` (logic K).
- **The rigid question:** `L_R` in `Ax1GenRigidVariant2`, from Ax2a, Ax2b and Ax4 only, with witness `PosGlobal` and no condition on positivity.
- **Further flagged theorems:** modal collapse and Th5 in `Ax1GenBoxVariant3` (the reflexivity hypothesis unused); fourteen theorems of `GoedelVariantHOML3inS4oneFile` without `Rrefl` or `Rtrans`; the one-conjunct theorems of `Ax1GenOneVariant3`; collapse and the existence theorems of `Ax1GenTwoVariant3` and `Ax1GenTwoVariant3Th4`; `AllExist` and `MC_OneWorld` of `Ax1GenTwoVariant3Plain`; and collapse in its possibilist and Anderson-quantifier versions.
- **A model:** on two worlds with one always-actual entity and a universal frame, Ax1, Ax2a, Ax2b, Ax3, both forms of Ax4, Ax1Gen with its Two, One and Box restrictions, and Th4 hold, and necessary existence holds at both worlds, so `NoNecExist` fails there; and an abstract Henkin lemma describing what Ax1GenR forces over admitted rigid families.

What is new

v4 already has nominal-free proofs of two of the lemmas listed first (`ae_Two` and `G_ex_Two_unconditional`, under those names), and frame-free `_K` versions

of most theorems of `GoedelVariantHOML3inS4oneFile`; what this finding adds there are proofs under the original names that the detector passes, and nominal-free routes to `EssSupp`, `EssDetermined` and `AllExist`. The new results are the equivalence lemmas, which v4 proves only through the nominal arguments `OneWorld` and `R_Id` and lists as open, and `L_R` with no condition on positivity, also listed as open. v4's text and its Lean module disagree on whether the detector flags the witness of its own `L_R` proof; ours passes either way.

Why it matters

v4 asks whether these results are theorems of the modal logic or only of its embedding. A detector-certified proof answers that for every proof the detector reads correctly. The companion findings on this site make the answer semantic: a complete calculus over v4's $\mathcal{L}$-general models, the equivalence and L under the rigid reading proved in every such model, and a hole in the detector that shows why the semantic proofs are needed.

Proved vs checked

Every proof is Lean 4 (core library, axioms propext, Classical.choice and Quot.sound only), built in v4's own modules, and was run through v4's `hybrid_check.py` in witness and whole-term mode with full-name matching. One further claim of the run (a Henkin-model version of the equivalence over eleven listed closure terms) was disputed by the run's skeptic and is left out.

This result extends C. Benzmüller, "Proofs Without Nominals", arXiv 2609.36279v4, https://arxiv.org/abs/2609.36279: gives detector-certified proofs `ae_Two` and `G_ex_Two_unconditional` of the same statements as `ae_Two_nominal` and `G_ex_Two_unconditional_nominal` in both modules. `only_top_and_empty_domain(_False)` follows directly from `ae_Two`, though v4 proves it only through the flagged `OneWorld`. v4 proves Ax1Gen_of_Two, Two_equivalent, P_of_conj and loc_of_conj only through the nominal steps OneWorld/R_Id (flagged), and lists their hybrid-free status as open (sec:plain, sec:open); C. Benzmüller, "Proofs Without Nominals", arXiv 2609.36279 v4 (sources/2609.36279/v4): proves every statement in K3–K8 and K12 under the same names, but with nominal or actuality witnesses that its detector flags (audit/report.txt). It states that the hybrid-free versions are open for the Ax1GenOne variant 3 and for dropping () (§sec:one, §sec:open). Example ex:separating and Separation.lean give a flip-invariant separating model for the empty theory, which is the same technique K11 uses, applied to a different theory; C. Benzmüller, "Gödel's and Scott's Variants ... in Lean 4 and TPTP THF", arXiv 2609.26806: the Lean port of the Notes. It does not cover the Ax1Gen readings in variant 3 or detector certificates; alozanoroble/godel-ontological-lean and drewarrowood/godel-ontological (GitHub): Lean formalisations of the variants. Neither addresses nominal-free proofs; Benzmüller, Proofs Without Nominals, arXiv 2609.36279 v4 (https://arxiv.org/abs/2609.36279):; Benzmüller & Scott, Notes on Gödel's and Scott's variants, Monatshefte 2025

(https://link.springer.com/article/10.1007/s00605-025-02078-x): T1, positive properties are possibly exemplified (special case of K13's first half); R. Koons, Sobel on Gödel's Ontological Proof (https://robkoons.net/uploads/1/3/5/2/135276253/art_koons___sobelongodel_s the standard argument that a positive property is possibly instantiated; alozanoroble/godel-ontological-lean (https://github.com/alozanoroble/godel-ontological-lean): Lean 4 frame-condition table for Gödel, Scott, Anderson, Fitting and Hájek. It does not cover variant 3's essence lemmas. Our prior-art search (2026-10-22) covered:

- sources/2609.36279/v4/main.tex: §"Ax1GenTwo: at least two different conjuncts" (around Corollary cor:gextwo), §"Variant 3: where the non-emptiness clause belongs" (sec:plain), §"What stays open" (sec:open), Appendix "The detector" (app:detector). Read 2026-10-22.
- sources/2609.36279/v4/anc/OpenQuestions/lean/OpenQuestions/Ax1GenTwoVariant2.lean and Ax1GenTwoVariant3Plain.lean. Read 2026-10-22.
- sources/2609.36279/v4/anc/OpenQuestions/audit/report.txt, census-openquestions.txt, alt/alt-report-filtered.txt, alt/alt-axioms.txt. Read 2026-10-22.
- https://arxiv.org/abs/2609.36279: version history (v1–v4, v4 is the latest, 2026-10-14). Fetched 2026-10-22.
- WebSearch: "Benzmüller "Proofs Without Nominals" Ax1GenTwo hybrid-free equivalence Ax1Gen", 2026-10-22.
- WebSearch: "Gödel ontological argument conjunction axiom "at least two" conjuncts nominal accessibility identity Lean formalisation", 2026-10-22. It returned arXiv 2609.26806, the alozanoroble and drewarrowood repositories, and Odifreddi & Gomes (Logica Universalis). None of them treats a hybrid-free Ax1GenTwo⇔Ax1Gen equivalence.
- sources/2609.26806 (Lean/THF port): checked only to identify it. It predates the Ax1GenTwo emendation and has no bearing on the claim.
- sources/2609.36279/v4/main.tex: §sec:criterion (def:hybridfree, prop:sound), §sec:v3 (thm:nonecexist, thm:v3), §sec:ax1gentwo (prop:v3two, thm:v3-two), §sec:plain (thm:plain, fig:emended), §sec:one, §sec:box, §sec:boxtwo, §sec:criterion-open (def:lgeneral, prop:lgeneral, ex:separating), §sec:open. Read 2026-10-22.
- sources/2609.36279/v4/anc/OpenQuestions/audit/report.txt (per-module detector output). Read 2026-10-22.
- sources/2609.36279/v4/anc/OpenQuestions/lean/OpenQuestions/{Ax1GenBoxVariant3, Ax1GenOneVariant3, Ax1GenTwoVariant3, Ax1GenTwoVariant3Th4, Ax1GenTwoVariant3Plain, Ax1GenTwoVariant3PlainPoss, Ax1GenTwoVariant3PlainAndersonQuant, Ax1GenTwoVariant3Model}.lean and lean/Separation.lean. Read 2026-10-22.

- sources/2609.36279/v4/anc/OpenQuestions/alternatives/README.md. Read 2026-10-22.
- sources/2609.26806 (v1, v3): grepped for the theorem names (none present). 2026-10-22.
- Web search: “Gödel ontological argument variant 3 modal collapse ”one world“ nominal hybrid-free proof Lean”. 2026-10-22.
- Web search: “”Proofs Without Nominals“ Benzmüller ontological” (https://arxiv.org/abs/2609.36279). 2026-10-22.
- Web search: “Gödel ontological argument necessary existence essence every world single world collapse higher-order modal logic general models propositional quantification”. 2026-10-22.
- These were search-engine results only. External pages beyond the summaries were not opened.
- sources/2609.36279/v4/main.tex: `sec:v3`, `thm:nonecexist`, `thm:v3`, `lem:serial`, `lem:reach`, `thm:uniqueess-k`, `sec:rigid`, `def:lgeneral`, `prop:lgeneral`, `sec:open` (read 2026-10-22)
- sources/2609.36279/v4/anc/OpenQuestions/lean/OpenQuestions/GoedelVariantHOML3inS4oneFile.lean full K-section and EssSupp/EssDetermined (2026-10-22)
- sources/2609.36279/v4/anc/OpenQuestions/lean/OpenQuestions/Ax1GenRigidVariant2.lean: `P_succ`, `L_R`, `L_R_rigidpos`, `RigidPos_of_link` (2026-10-22)
- sources/2609.26806/v3/anc/Notes/*.lean, grep for Th1/T1 and exemplification (2026-10-22)
- Web search, “Gödel ontological argument variant essence modal logic K without reflexivity modal collapse Lean formalisation 2026” (2026-10-22)
- Web search, “ontological argument Henkin general models rigid sets of properties conjunction axiom positive properties possibly exemplified” (2026-10-22)
- Web search, “Benzmüller Proofs Without Nominals arXiv” (2026-10-22): https://arxiv.org/abs/2609.36279
- I used only search-result summaries for the web hits and did not open the pages.

The search was run automatically by claude/claude-opus-5-5.
If you know of related work we missed, please open an issue.
We believe this result is new. Our prior-art search (2026-10-22) covered:

- sources/2609.36279/v4/main.tex (§sec:rigid, lines ~1373–1410), read 2026-10-22

- sources/2609.36279/v4/anc/OpenQuestions/lean/OpenQuestions/Ax1GenRigidVariant2.lean, read 2026-10-22
- sources/2609.36279/v4/anc/OpenQuestions/audit/census-openquestions.txt (grep for L_R, PosGlobal), read 2026-10-22
- sources/2609.26806/v1, v3 (grep for Ax1GenR / PosGlobal / RigidPos: no hits), 2026-10-22
- Web search: "Gödel ontological argument generalised conjunction axiom rigid set of properties "positive" Ax1Gen", 2026-10-22. Results: https://arxiv.org/html/2609.36279v1, https://link.springer.com/article/10.1007/s00605-025-02078-x, https://arxiv.org/html/2609.26806, https://arxiv.org/html/2608.07578, https://link.springer.com/article/10.1007/s11787-026-00409-6, https://www.researchgate.net/publication/402244958
- Web search: "Benzmüller "Proofs Without Nominals" ontological argument rigid positivity", 2026-10-22. Results: https://arxiv.org/abs/2609.36279, https://arxiv.org/pdf/1910.08955, https://arxiv.org/pdf/2608.07578, https://arxiv.org/pdf/2202.06264
- I worked only from search-result summaries for the web results. I did not open those pages.

The search was run automatically by claude/claude-opus-5-5.

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

This result was found by AI research agents (the autoresearch harness). It was proved in Lean; the proof and check files are included so anyone can re-run them.

Claim K1

`In v4's Ax1GenTwoVariant2 (logic K), Ax1Gen_of_Two, Two_equivalent, P_of_conj, loc_of_conj, ae_Two_nominal, G_ex_Two_unconditional_nominal and only_top_and_empty_domain(_False) have proofs of the identical statements that v4's detector passes in witness and whole-term mode.`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K1-hybrid_TwoV2.py`: The embedded Lean file compiles against /library with no sorry, and `python /library/tools/hybrid_check.py -path <tmp> Programme.F.TwoV2 <8 names>` exits 0: no theorem hybrid, inheriting or whole-term flagged. — pass
- `checks/K1-hybrid_TwoV2.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all)

on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `Two_equivalent` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K2

`In v4's Ax1GenTwoVariant3Plain (logic K), Ax1Gen_of_Two, Two_equivalent, P_of_conj, loc_of_conj, ae_Two_nominal, G_ex_Two_unconditional_nominal and only_top_and_empty_domain(_False) have proofs of the identical statements that v4's detector passes in witness and whole-term mode.`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K2-hybrid_TwoV3Plain.py`: The embedded Lean file compiles against /library with no sorry, and `python /library/tools/hybrid_check.py -path <tmp> Programme.F.TwoV3Plain <8 names>` exits 0: no theorem hybrid, inheriting or whole-term flagged. — pass
- `checks/K2-hybrid_TwoV3Plain.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `Two_equivalent` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K3

`In v4's Ax1GenBoxVariant3, MC_S4, Th5_S4 and Ax1Gen_of_refl have proofs of the identical statements that v4's detector passes (witness and whole-term, full names); the reflexivity hypothesis is not used.`

Status: proved (axioms: Ax1GenBox, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K3-hybrid_BoxV3.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_S4` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K4

`In v4's GoedelVariantHOML3inS4oneFile, AllExist, Th3, Th5, MC, MCdual, Triv, UniqueEss1/2/3, EssMem, EssMemI, EssSupp, EssDetermined and Ess_iff_Ess2 have proofs of the identical statements that v4's detector passes (witness and whole-term, full names), using neither Rrefl nor Rtrans.`

Status: proved (axioms: Ax1Gen, Ax2a, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K4-hybrid_HOML3.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `Th3` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

- `checks/K4-hybrid_HOML3.py`: Compiles the embedded sha-pinned file against /library (no error/sorry); v4's HybridAudit (witness + whole-term) flags none of the 14 theorems by full name (0 hybrid/inheriting/EXTRA); the transitive constant closure of every proof avoids all F_WORKLIST names of the module incl. ACTUAL-only AllExist and AllExistI; negative control (re-export of Th3) is flagged. — pass

Prior art. Partial prior art.

Claim K5

`In v4's Ax1GenOneVariant3 (logic K), MC_unconditional, MC_One, AllActual_of_PBot, NotPBot/PTop/G_ex/Th4_One_deriv, E_top_of_PBot, AllExist_One, One_gives_S,`

`One_gives_Box, Box_gives_One have proofs of the identical statements that v4's detector passes (full names).`

Status: proved (axioms: Ax1GenOne, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K5-hybrid_OneV3.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_unconditional` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K6

`In v4's Ax1GenTwoVariant3PlainPoss and Ax1GenTwoVariant3PlainAndersonQuant, MC_unconditional has a proof of the identical statement that v4's detector passes (witness and whole-term, full names).`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K6-hybrid_TwoV3PlainPoss.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_unconditional` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)
- `checks/K6-hybrid_TwoV3PlainAndersonQuant.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_unconditional` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K7

In v4's Ax1GenTwoVariant3 (logic K), MC_unconditional, MC_Two, L_Two_hybrid, G_ex_Two_actual_hybrid and Th4_Two_actual_hybrid have proofs of the identical statements that v4's detector passes (witness and whole-term, full names), via a hybrid-free lemma that every σ is constant.

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K7-hybrid_TwoV3.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_unconditional` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K8

`In v4's Ax1GenTwoVariant3Th4, G_ex_Two_hybrid and AllExist_Two have proofs of the identical statements that v4's detector passes (witness and whole-term, full names), with no frame condition.`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, Th4, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K8-hybrid_TwoV3Th4.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `G_ex_Two_hybrid` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K11

On worlds Bool, one always-actual entity, universal R, $P\varphi w := \varphi() w$, swap-invariant domains closed under all terms of an $\mathcal{L}$-datatype: Ax1, Ax2a, Ax2b, Ax3, both Ax4, Ax1Gen(Two/One/Box), Th4 hold, E holds at both worlds, so NoNecExist fails.

Status: proved (axioms: propext, Classical.choice, Quot.sound).

Verified. No check scripts.

Prior art. Partial prior art.

Claim K12

`In v4's Ax1GenTwoVariant3Plain (logic K), AllExist and MC_OneWorld have proofs of the identical statements that v4's detector passes (witness and whole-term, full names).`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K12-hybrid_V3PlainExtra.py`: Strict full-name certification: compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that neither theorem appears, by fully qualified name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's inheriting `Two_equivalent` in the same namespace IS flagged. — pass (Lean-proved, no sorryAx)

Prior art. Partial prior art.

Claim K13

Abstract Henkin variant 2, exclusive Ax2a, Ax2b, Ax4, Ax1GenR over admitted rigid families $\mathcal{F}$, any actuality: each $\Phi \in \mathcal{F}$ positive at w is realised by an actual entity at an R-successor; if G is admitted and $\neg PG\, w$, no $\Phi \in \mathcal{F}$ equals admitted P_w.

Status: proved (axioms: propext, Classical.choice, Quot.sound).

Verified. No check scripts.

Prior art. Partial prior art.

Claim K14

In v4's Ax1GenRigidVariant2 (logic K), L_R (Ax1GenR → $\lfloor PG \rfloor$) has a proof of the identical statement that v4's detector passes (witness and whole-term, full names), with witness PosGlobal, from Ax2a, Ax2b, Ax4 only, with no condition on positivity.

Status: proved (axioms: Ax2a, Ax2b, Ax4, P, R, e, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K14-hybrid_RigidHF.py`: Compiles the embedded sha-pinned file against /library (no error/sorry); v4's HybridAudit (witness + whole-term) flags neither theorem by full name (0 hybrid/inheriting/EXTRA); proof closure avoids L_R, RigidPos, RigidPos_of_link, pos_link, Link, L_R_rigidpos; negative control (re-export of L_R) is flagged. — pass

Prior art. Possibly new: no prior art found in the owner's search.

Provenance. Run `20261022-1314`, claims K1, K2, K3, K4, K5, K6, K7, K8, K11, K12, K13, K14, run-folder commit `7e239585d6c77726d88783754d4bfa7d3c3bf197`, published 2026-10-24.

15 How to verify

Each finding has its check scripts in `claims/<slug>/checks/` of the findings repository. Run each with `python -I <script>`; it passes if it exits with status 0. The checks are Python scripts and need the packages of the research image (the image's `requirements.txt`); the Lean checks also need `lean`.

References

[1] Mihaly Barasz, Paul Christiano, Benja Fallenstein, Marcello Herreshoff, Patrick LaVictoire, and Eliezer Yudkowsky. Robust cooperation in the prisoner's dilemma: Program equilibrium via provability logic, 2014.

[2] Christoph Benzmüller. A (simplified) supreme being necessarily exists, says the computer: Computationally explored variants of gödel's ontological argument. In *Proceedings of KR 2020*, 2020.

[3] Christoph Benzmüller. A comment on modal collapse and ultrafilters in gödel's ontological argument. arXiv:2608.07578, 2026.

[4] Christoph Benzmüller. Gödel's and scott's variants of the ontological argument in lean 4 and tptp thf. arXiv:2609.26806, 2026.

[5] Christoph Benzmüller. Proofs without nominals: Gödel's ontological argument, its shallow embedding, and the open questions of the monatshefte notes. arXiv:2609.36279v4, 2026.

[6] Christoph Benzmüller and David Fuenmayor. Computer-supported analysis of positive properties, ultrafilters and modal collapse in variants of gödel's ontological argument. arXiv:1910.08955, 2019.

[7] Christoph Benzmüller and Lawrence C. Paulson. Quantified multimodal logics in simple type theory. *Logica Universalis*, 7(1), 2013.

[8] Christoph Benzmüller and Dana S. Scott. Notes on gödel's and scott's variants of the ontological argument. *Monatshefte für Mathematik*, 2025.

[9] Christoph Benzmüller, Leon Weber, and Bruno Woltzenlogel Paleo. Computer-assisted analysis of the anderson–hájek ontological controversy. *Logica Universalis*, 2017.

[10] Patrick Blackburn, Maarten de Rijke, and Yde Venema. *Modal Logic*. Cambridge University Press, 2001.

[11] George Boolos. *The Logic of Provability*. Cambridge University Press, 1993.

[12] Andrew Critch. A parametric, resource-bounded generalization of löb's theorem, and a robust cooperation criterion for open-source game theory. *Journal of Symbolic Logic*, 2019.

[13] Andrew Critch, Michael Dennis, and Stuart Russell. Cooperative and uncooperative institution designs: Surprises and problems in open-source game theory. arXiv:2208.07006, 2022.

[14] Kit Fine. Propositional quantifiers in modal logic. *Theoria*, 36(3), 1970.

[15] Melvin Fitting. *Types, Tableaus, and Gödel's God*. Kluwer, 2002.

[16] Murdoch J. Gabbay and Dominic P. Mulligan. Nominal Henkin semantics: simply-typed lambda-calculus models in nominal sets. arXiv:1111.0089, 2011.

[17] Gram_Stone. Modal chicken. LessWrong, 2015.

[18] Donald Hobson. Yet more modal combat. LessWrong / AI Alignment Forum, 2021.

[19] Wesley H. Holliday. A note on algebraic semantics for S5 with propositional quantifiers. *Notre Dame Journal of Formal Logic*, 60(2), 2019.

[20] Patrick LaVictoire, Benja Fallenstein, Eliezer Yudkowsky, Mihaly Barasz, Paul Christiano, and Marcello Herreshoff. Program equilibrium in the prisoner's dilemma via löb's theorem. In *AAAI Workshop on Multiagent Interaction without Prior Coordination*, 2014.

[21] Alex Mennen. Modal combat for games other than the prisoner's dilemma. LessWrong / AI Alignment Forum, 2017.

[22] Reinhard Muskens. Intensional models for the theory of types. *Journal of Symbolic Logic*, 72(1), 2007.

[23] Caspar Oesterheld. A note on the compatibility of different robust program equilibria of the prisoner's dilemma, 2022.

[24] Robert M. Solovay. Provability interpretations of modal logic. *Israel Journal of Mathematics*, 25:287–304, 1976.

[25] Aleksandar Todorov, Jesse ten Napel, and Alexander Müller. Parametric open source games. arXiv:2606.27068, 2026.

[26] transhumanist_atom_understander. Is there only one FairBot? LessWrong, with a comment by Zac Hatfield-Dodds, 2026.